

注意事項：本資料の再利用 (2 次利用) について

● 本資料について

- 東京大学 松尾・岩澤研究室が作成し、2025 年 10 月から 11 月にかけて開催された[LLM大規模言語モデル講座基礎編](#)の講義資料です。
- クリエイティブ・コモンズの [CC BY-NC-SA 4.0 DEED](#)(表示 – 非営利 – 継承 4.0 国際) のライセンス登録をしています。

● ライセンスの表示について

- 各スライドのページ最下部にライセンスの記載があります。再利用時には本ライセンス表示を必ずご記載ください。再利用時に複製が困難な場合は、下記のテキストボックスを利用の上、ハイパーリンクも含めてライセンスの表記をするようお願いいたします。

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND 4.0

- 再利用するページに参照論文等の引用がある場合は、巻末にある Reference より引用箇所を掲載してください。

● 非営利目的での利用について 再利用 (2 次利用) が許諾されています。

● 営利目的での再利用について [こちら](#)からお問い合わせください。

● その他

- 元の表現が変わらない範囲 (フォント、サイズ等) であれば改変可能です。
- それ以外の改変その他ライセンスについての詳細は、[こちら](#)をご覧ください。

5. Advanced Pre-training

2025/10/29

山田 育矢（やまだ いくや）

Studio Ousia チーフサイエンティスト

名古屋大学 数理・データ科学・人工知能教育研究センター 客員教授

東北大学 言語 AI 研究センター 特任教授（客員）

博士（学術）



<https://ikuya.net>

ikuya@ikuya.net

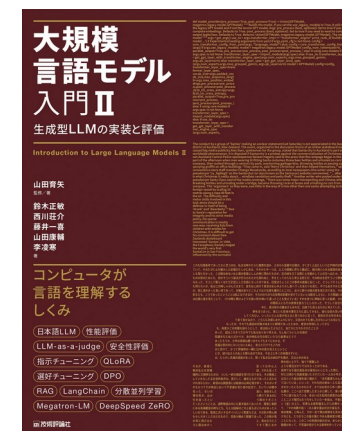
ikuyamada

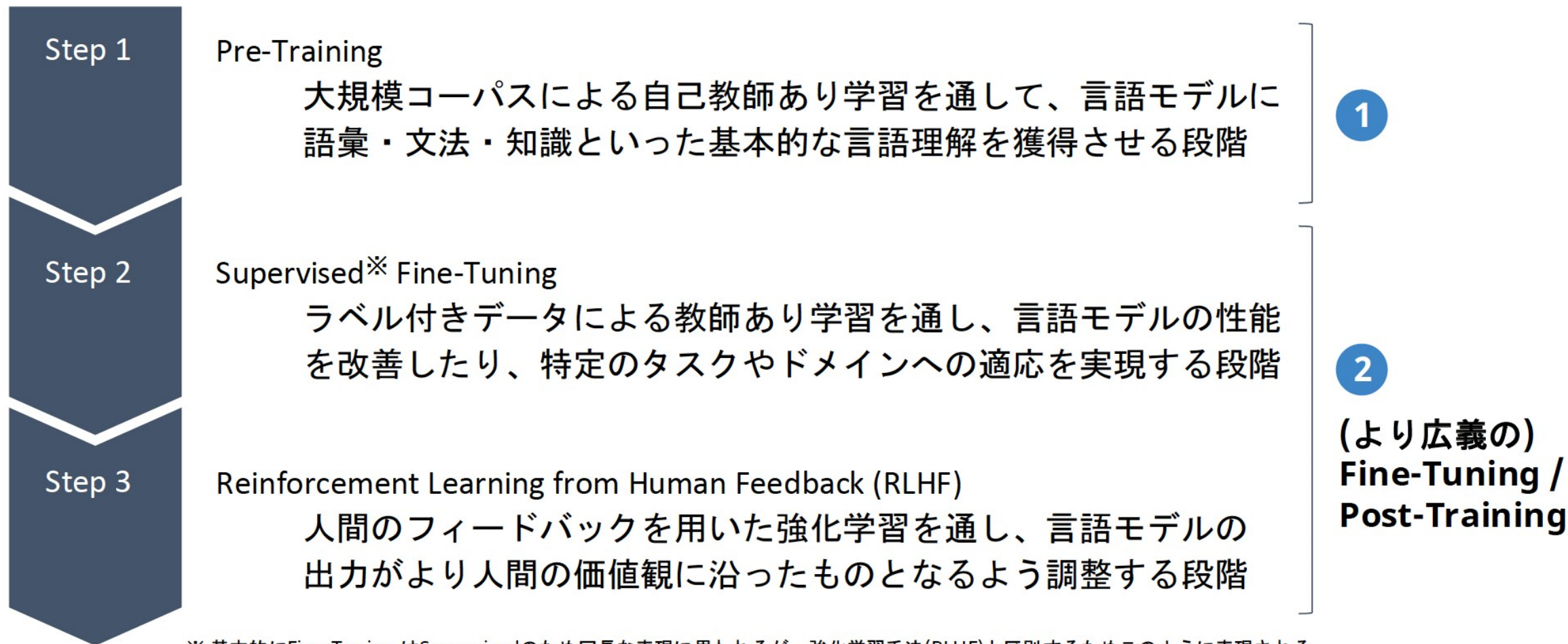
主な実績：

- 様々な言語モデルの開発
- 開発した早押しクイズ AI が全米クイズ王チームに勝利（NIPS Competition 2017）
- 複数の国際コンペティションで好成績を獲得
NeurIPS EfficientQA 2020 (2 位), ISWC Challenge 2020 (1 位),
NIPS HCQA 2017 (1 位), WSDM Cup 2017 (2 位),
NAACL HCQA 2016 (1 位), ACL W-NUT 2015 (1 位), etc.

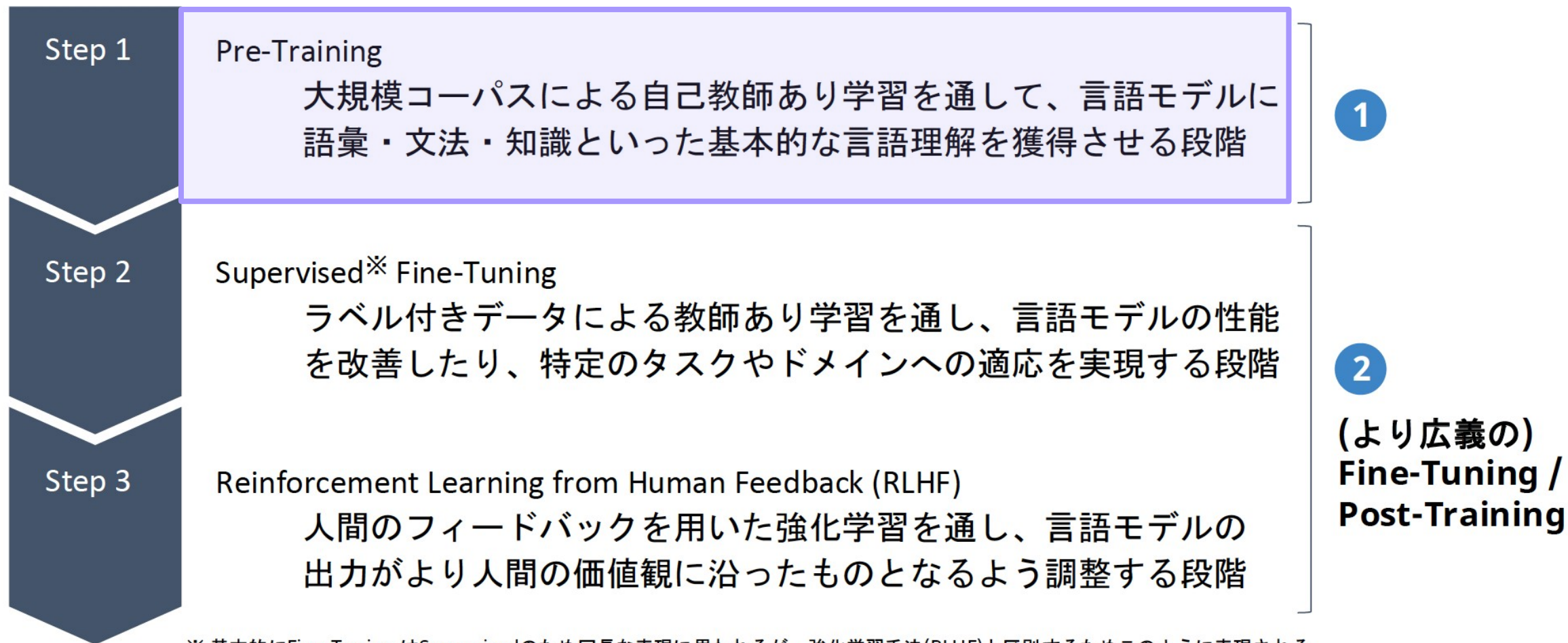
主な著書：

- 大規模言語モデル入門・大規模言語モデル入門 II
- ディープラーニングによる自然言語処理





※ 基本的にFine-Tuning はSupervisedのため冗長な表現に思われるが、強化学習手法(RLHF)と区別するためこのように表現される。
また、あえてこのように表現する場合には、一般の教師ありFine-Tuningではなく、後述のInstruction Tuningを指すことが多い。



※ 基本的にFine-Tuning はSupervisedのため冗長な表現に思われるが、強化学習手法(RLHF)と区別するためこのように表現される。
また、あえてこのように表現する場合には、一般の教師ありFine-Tuningではなく、後述のInstruction Tuningを指すことが多い。

1 Pre-Training

2 Fine-Tuning / Post-Training

目的

- 語彙・文法・知識・推論能力などの言語能力を、言語モデルに導入

- 事前学習済みモデルの性能改善や、様々なタスクに対する適応を実現

一般的な手法

- 自己教師あり学習
 - Next Token Prediction
 - Masked Language Model

- 教師あり学習
 - 下流タスクへの特化
 - Instruction Tuning

- 強化学習

データ

- 大規模データセット
 - 例 CommonCrawl (GPT-3): 410B tokens (570GB)

- 良質な小規模データセット
 - 例 LIMA: 1000サンプル (3MB)

- 人間・モデルによるフィードバック

第3回と今日のトピック

1 Pre-Training	2 Fine-Tuning / Post-Training
目的 - 語彙・文法・知識・推論能力などの言語能力を、言語モデルに導入	- 事前学習済みモデルの性能改善や、様々なタスクに対する適応を実現
一般的な手法 - 自己教師あり学習 - Next Token Prediction - Masked Language Model	- 教師あり学習 - 下流タスクへの特化 - Instruction Tuning - 強化学習
データ - 大規模データセット - 例 CommonCrawl (GPT-3): 410B tokens (570GB)	- 良質な小規模データセット - 例 LIMA: 1000サンプル (3MB) - 人間・モデルによるフィードバック

事前学習はどのくらい重要？



	team	size
0	Pretraining	34.529762
1	Long context	3.958333
2	Vision	22.298810
3	Reinforcement Learning & Alignment	43.395238
4	Evaluation & analysis	78.706349
5	Deployment	51.883333
6	Additional contributions	49.228175

[66] iwiwi, github,
<https://gist.github.com/iwiwi/fc174b1f2341c2c0170be87c5b2e1d31>,

GPT-4 の各開発タスクにおける人数が全体に占める割合 =
OpenAI の考える LLM 開発における重要度の割合！？ ☒

GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale language model that can accept image and text inputs and produce text comparable to humans in many real-world scenarios. GPT-4 excels on various professional and academic benchmarks, including a simulated bar exam with a score around the top 10% of test-takers. The model is pre-trained on a large corpus of text and fine-tuned using a reinforcement learning process results in improved performance and adherence to desired behavior. A core component of the development is the infrastructure and optimization methods that bridge a wide range of scales. This allowed us to accurately predict performance based on models trained with no more than 100M parameters.

Authorship, Credit Attribution, and Acknowledgments

Please cite this work as “OpenAI (2023)”.

Pretraining

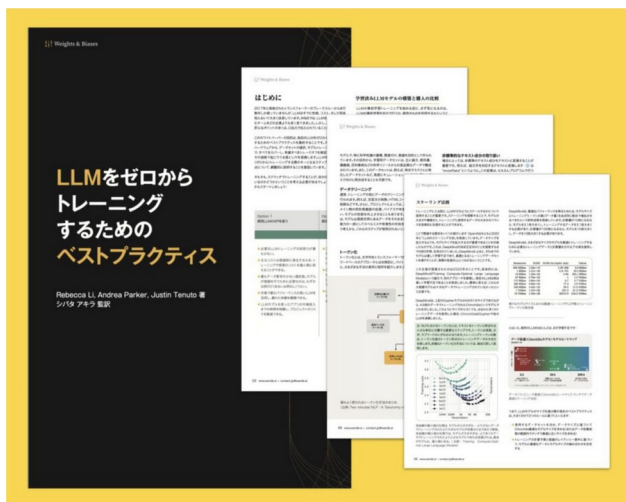
Core contributors¹¹

Christopher Berner *Supercomputing lead*
Greg Brockman *Infrastructure lead*
Trevor Cai *Throughput lead*
David Farhi *Manager of optimization team*
Chris Hesse *Infrastructure usability co-lead*
Shantanu Jain *Infrastructure usability co-lead*
Kyle Kosic *Uptime and stability lead*
Jakub Pachocki *Overall lead, optimization lead*
Alex Paino *Architecture & data vice lead*
Mikhail Pavlov *Software correctness lead*
Michael Petrov *Hardware correctness lead*
Nick Ryder *Architecture & data lead*
Szymon Sidor *Optimization vice lead*
Nikolas Tezak *Execution lead*
Phil Tillet *Triton lead*
Amin Tootoonchian *Model distribution, systems & networking lead*
Qiming Yuan *Dataset sourcing and processing lead*
Wojciech Zaremba *Manager of dataset team*

Compute cluster scaling¹¹

Christopher Berner, Oleg Boiko, Andrew Cann, Ben Chess, Christian Gibson, Mateusz Litwin, Emy Parparita, Henri Roussez, Eric Sigler, Akila Welihinda

重要な問い：事前学習を自前で実施する必要があるか？ (1)



Option 1 商用LLMのAPIを使う

- 必要なLLMトレーニングの技術力が最も少ない。
- 主なコストは推論時に発生するため、トレーニングや探索のコストを最小限に抑えることができる。
- 最もデータ要求の少ない選択肢。モデルが推論を行うために必要なのは、わずかな例だけ（あるいは例なし）でよい。
- 市場で最もパフォーマンスの高いLLMを活用し、優れた体験を構築できる。
- LLMモデルを使ったアプリの市場投入までの時間を短縮し、プロジェクトのリスクを軽減できる。

Option 2 既存のオープンソースLLMを利用する

- ①と比較すると、LLMサービスプロバイダーの将来の方向性への依存度が低い。そのため、ロードマップや後方互換性をコントロールしやすい。
- ③と比較すると、LLMをゼロから構築しないため、データ、トレーニング時間、トレーニング予算が少なく済み、Time-to-Valueが大幅に短縮される。
- LLMが膨大なインターネットデータから学んだことを活用し、推論時に知財にお金を払うことなく、構築できる。

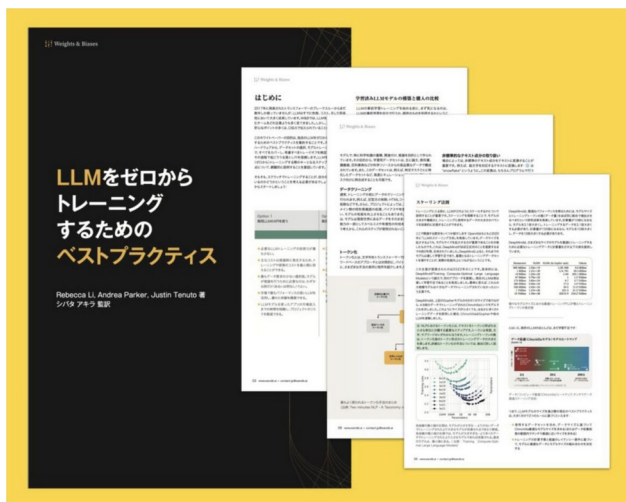
Option 3 自らもしくは外部支援を受けてLLMをトレーニングする

- ①、②に比べ、LLMのパフォーマンスと将来の方向性を最もコントロールできるため、技術革新や下流タスクへのカスタマイズの自由度が高い。
- モデルの品質、バイアス、有害性の問題に直接影響する学習用トレーニングデータセットを完全にコントロールすることができる。これに対し、①や②では、これらの問題をコントロールすることは不可能。
- LLMをトレーニングすることで、強固な優位性を築くことができる。水平方向のユースケースや垂直方向に合わせたLLMの優れたパフォーマンスにより、特にLLMの展開を可能にするデータ/フィードバックループを作成した場合、持続的な優位性を構築することができる。

長 所

[65] weights & biases, 「LLM をゼロからトレーニングするためのベストプラクティス」より引用

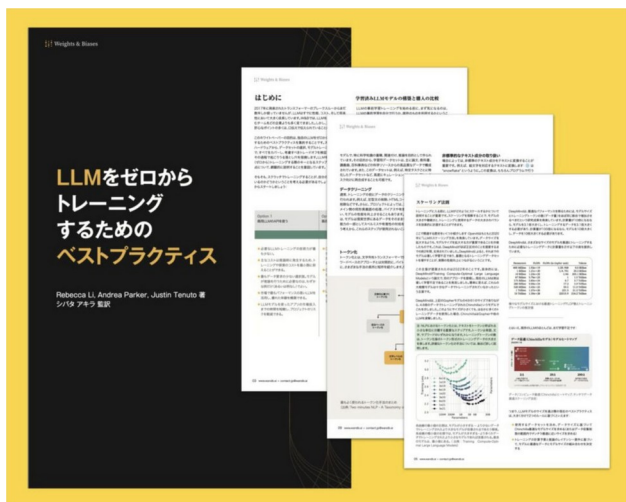
重要な問い：事前学習を自前で実施する必要があるか？ (2)



Option 1 商用LLMのAPIを使う	Option 2 既存のオープンソースLLMを利用する	Option 3 自らもしくは外部支援を受けてLLMをトレーニングする
短 所		
● 商用LLMサービスは、微調整や推論タスクが大量に発生すると高価になることがある。これは、LLMの総所有コスト(TCO)を各推論に償却したものです。 ● 多くの業界やユースケースでは、コンプライアンス上、機密データやPIIデータの閲覧を許可できないため、商用LLMサービスの利用を禁じている(例えば、ヘルスケアユースケース)。 ● 外部アプリを構築する場合、外部のLLMサービス技術への依存度が高い場合は、他の強みを探し、事業のリスクを軽減する必要がある。 ● 下流の柔軟性が低い: エッジ推論をサポートしていない、モデルをカスタマイズする機能が限られている(微調整をするとコストがかかる)、モデルを継続的に改善する機能が限られている。	● 自分で作る時ほどではないが、オープンソースのLLMをトレーニングし、微調整し、ホストするためには、多くのドメイン専門家のスキルが必要である。LLMの再現性は依然として重要な問題であるため、必要な作業量と時間は過小評価できない。 ● より垂直な技術スタックのため、下流のアプリを構築している場合、市場投入までの時間が遅く、アジャイル性が低い。 ● オープンソースのモデルは、通常、商用モデルと比較して数ヶ月から数年単位で性能が遅れている。競合他社が商用モデルを利用している場合、彼らはLLM技術で優位に立っている可能性があるため、他の競争優位性を見つける必要がある。	● 非常に高価で、リスクも高い。NLP/ML、専門知識、ソフトウェア、ハードウェアの専門知識など、領域横断的な知識が必要。うまくやらないと、最適でないモデルで何千ドル、何百万ドルも費やしてしまったという事態になりかねない。特にトレーニングの後半でのミスは、修正・解消するのが難しい。 ● ②より効率が悪い。②は、インターネット上の全データから学習した既存のLLMを活用し、確度の高い初期モデルを提供することができる。③では、ゼロから始めることになり、モデルが汎化性能を獲得するためには、高品質で多様なデータセットが膨大に必要。

[65] weights & biases, 「LLM をゼロからトレーニングするためのベストプラクティス」より引用

重要な問い：事前学習を自前で実施する必要があるか？ (3)



Option 1

商用LLMのAPIを使う

- 技術チームは少ないが、LLM技術を活用してダウンストリームアプリを構築したい、あるいはパフォーマンス上の理由からクラス最高のLLMを活用したい (LLM技術をアウトソーシングする) 場合に最適。
- トレーニングデータセットが非常に限られていて、LLMのゼロ/少数回ショットの学習機能を活用したい場合。
- アプリのプロトタイピングや、LLMで何が可能かを探索したい時。

Option 2

既存のオープンソースLLMを利用する

- ②と③の間では、モデルのアーキテクチャを変えようとしないのであれば、既存の学習済みLLMを使って微調整するか、既存の学習済みLLMのモデルを出発点として追加学習を行う方が良い場合がほとんどである。GPT-NeoXのような優れた学習済みLLMは、すでに膨大な量のデータから学習しているため、汎化性能を獲得している。特に手元のトレーニングデータセットが巨大でなく、多様でない場合、その学習を活用することができる。
- 規制の厳しい環境にいたり、ユーザーデータや機密データがあり、市販のLLMサービスには対応できない場合。また、レイテンシーや場所の制約があり、モデルのエッジ展開が必要な場合もある。

Option 3

自らもしくは外部支援を受けてLLMをトレーニングする

- 既存の学習済みLLMからモデルアーキテクチャや学習データセットを変更する必要がある場合に最適。例えば、異なるトークナイザーを使用したい場合、語彙サイズを変更したい場合、隠れ次元、アテンションヘッド、層の数を変更したい場合など。
- 一般的に、この場合、LLMはビジネス戦略や技術的な優位性の中核をなすという前提がある。LLMトレーニングの一部または多くのイノベーションを目的とし、高価なモデルを継続的にトレーニングし、維持するための莫大な投資計画がある場合。
- LLMサービスに関連して蓄積された多くの独自データを持ち、持続的な競争優位のための継続的なモデル改善ループを構築している、またはする予定がある場合です。

各選択肢を検討するタイミング

[65] weights & biases, 「LLM をゼロからトレーニングするためのベストプラクティス」より引用

スケール則：言語モデルをスケール（大規模化）させることで性能が向上する関係

以下の3つの要素と性能 (L) の間で成り立つ経験則

■ 計算資源 (C)

■ データセットサイズ (D)

■ パラメータ数 (N)

- 様々なドメインで大規模なモデルを開発する利点を確認された。
- スケール則により、大規模なモデルへの投資リスクが軽減された。

Advanced Pre-training の目的



(* 第 3 回の続き)

言語モデルを **スケール (=大規模化)** して事前学習することの発展的な話題について学ぶ。

Goal 1

モデルをスケールして事前学習する (発展的な) **方法** について説明できる。

Goal 2

モデルをスケールして事前学習する上での (発展的な) **課題** について説明できる。

Goal 3

事前学習の一連の流れをコードで **実装** できる (モデルをスケールするための技術も含む) 。

- 講義:

- それぞれの要素のスケールにおける問題点
- スケールするための技術

- 演習:

- PyTorch で Transformer モデルを事前学習するための一連の流れを実装
(データの準備, 前処理からモデルをスケールする技術を使った学習まで)

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

- **それぞれの要素のスケールにおける問題点**
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

パラメータ, 計算量, データをスケールする事で,
スケール則に従って性能が上がることはわかったが,
スケールさせる上で様々な課題がある

パラメータ数 (N)
モデルがスケール
するにつれて増加する
コストを抑える必要

計算量 (C)
十分な計算量 /
メモリ量を確保して
効率よく訓練する必要

データ (D)
性能を発揮させるため
の学習用データを用意
する必要

N, C : モデルサイズの増加につれて必要なコストが増加

“Mosaic LLMs (Part 1): Billion-Parameter GPT Training Made Easy”^[14] より抜粋

Model	# A100-40GB	Throughput (tokens/sec)	Hours / 1B Tokens	Multi-node speedup	Price / 1B Tokens		
					Oracle [BM.GPU4.8]	AWS [p4d.24xlarge] on-demand	AWS [p4d.24xlarge] spot
GPT3-125M	8	191,693	1.45	1.00x	\$35.36	\$47.49	\$14.24
GPT3-125M	32	739,328	0.38	3.86x	\$36.67	\$49.25	\$14.77
GPT3-125M	128	2,471,936	0.11	12.90x	\$43.87	\$58.92	\$17.67
GPT3-350M	8	69,427	4.00	1.00x	\$97.62	\$131.11	\$39.33
GPT3-350M	32	269,926	1.03	3.89x	\$100.44	\$134.89	\$40.46
GPT3-350M	128	958,464	0.29	13.81x	\$113.14	\$151.96	\$45.58
GPT3-760M	8	57,549	4.83	1.00x	\$117.77	\$158.17	\$47.45
GPT3-760M	32	218,317	1.27	3.79x	\$124.18	\$166.78	\$50.03
GPT3-760M	128	696,320	0.40	12.10x	\$155.74	\$209.16	\$62.74
GPT3-1.3B	8	38,912	7.14	1.00x	\$174.18	\$233.93	\$70.17
GPT3-1.3B	32	152,781	1.82	3.93x	\$177.45	\$238.32	\$71.49
GPT3-1.3B	128	561,152	0.50	14.42x	\$193.25	\$259.55	\$77.86



Table 2: Throughput metrics, multinode speedups, and estimated cost to train (per 1B tokens of data) for models in the GPT3 family. With good multinode scaling, we are able to greatly reduce training times with only a minimal increase in the total compute cost.

[14] Abhinav Venigalla, Linden Li, Billion-Parameter GPT Training Made Easy, MosaicML より引用

→ 効率的に大規模なモデルを訓練できれば、コストを減らすことができる

N, C: Transformer は系列長に対し必要な計算量 / メモリが増加

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

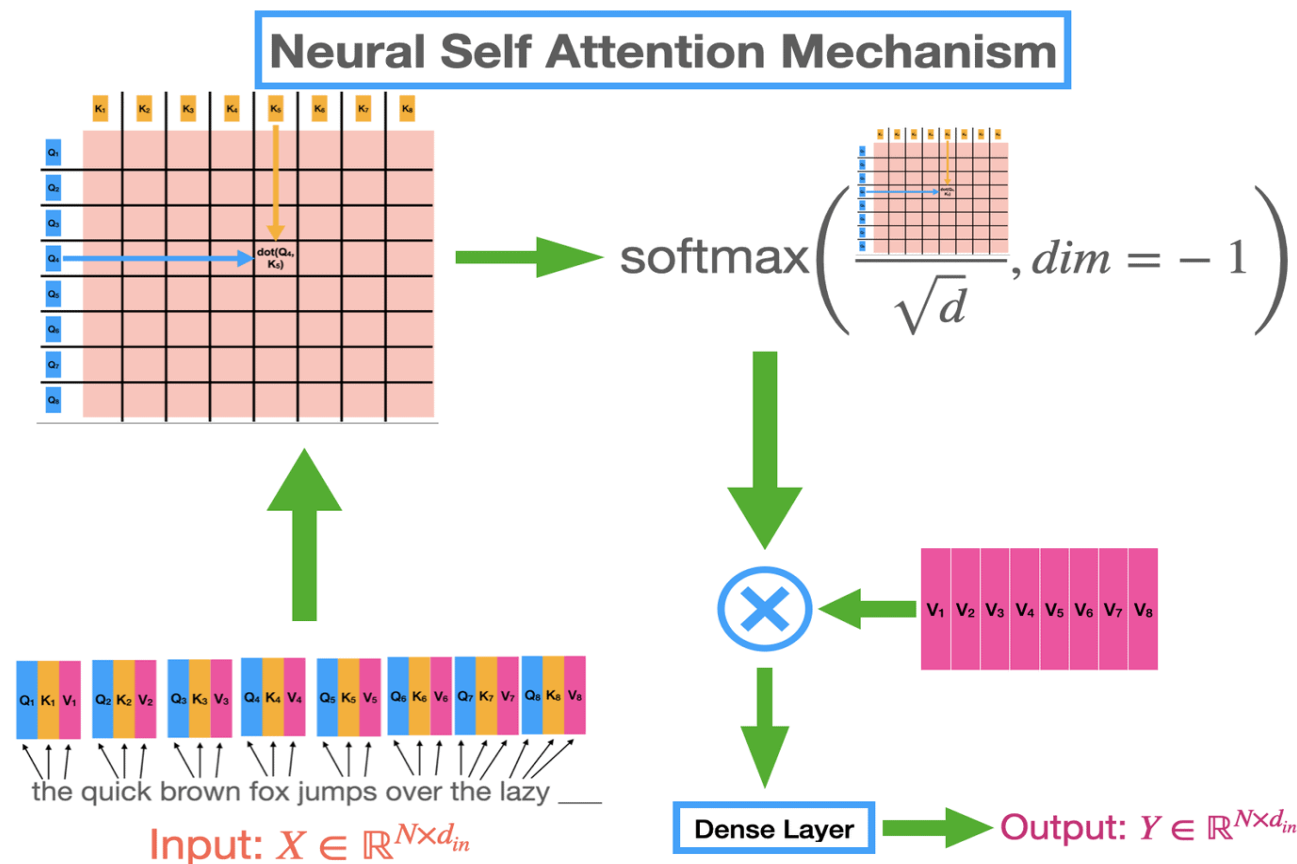
[15] Vaswani+. Attention Is All You Need. 2017 より引用

Self-Attention では，系列長 n の 2 乗の計算量とメモリが必要になる

■ 復習 | なぜ系列長の 2 乗の計算量 / メモリが必要なのか？



“Understanding Attention Mechanism in Transformer Neural Networks”^[16] より抜粋



[16] Jaiyam Sharma, Understanding Attention Mechanism in Transformer Neural Networks, LearnOpenCV より引用

各トークンが他のすべてのトークンとの関連性を計算するため、
全トークンの組み合わせに対して計算を行い、その値を記憶する必要がある。

	Historical projection	Compute projection
Low-quality language stock	2032.4 [2028.4 ; 2039.2]	2040.5 [2034.6 ; 2048.9]
High-quality language stock	2024.5 [2023.55 ; 2025.75]	2024.1 [2023.2 ; 2025.3]
Vision stock	2046 [2037 ; 2062.8]	2038.8 [2032.0 ; 2049.8]

TABLE IV: Median and 90% CI of exhaustion year for each of the intersections.

[17] Villalobos+. Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning. 2022 より引用

過去の Web データの増え方，学習データの増え方からの予測
良質な言語データは 2024 年頃に枯渇することが予測されている。

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

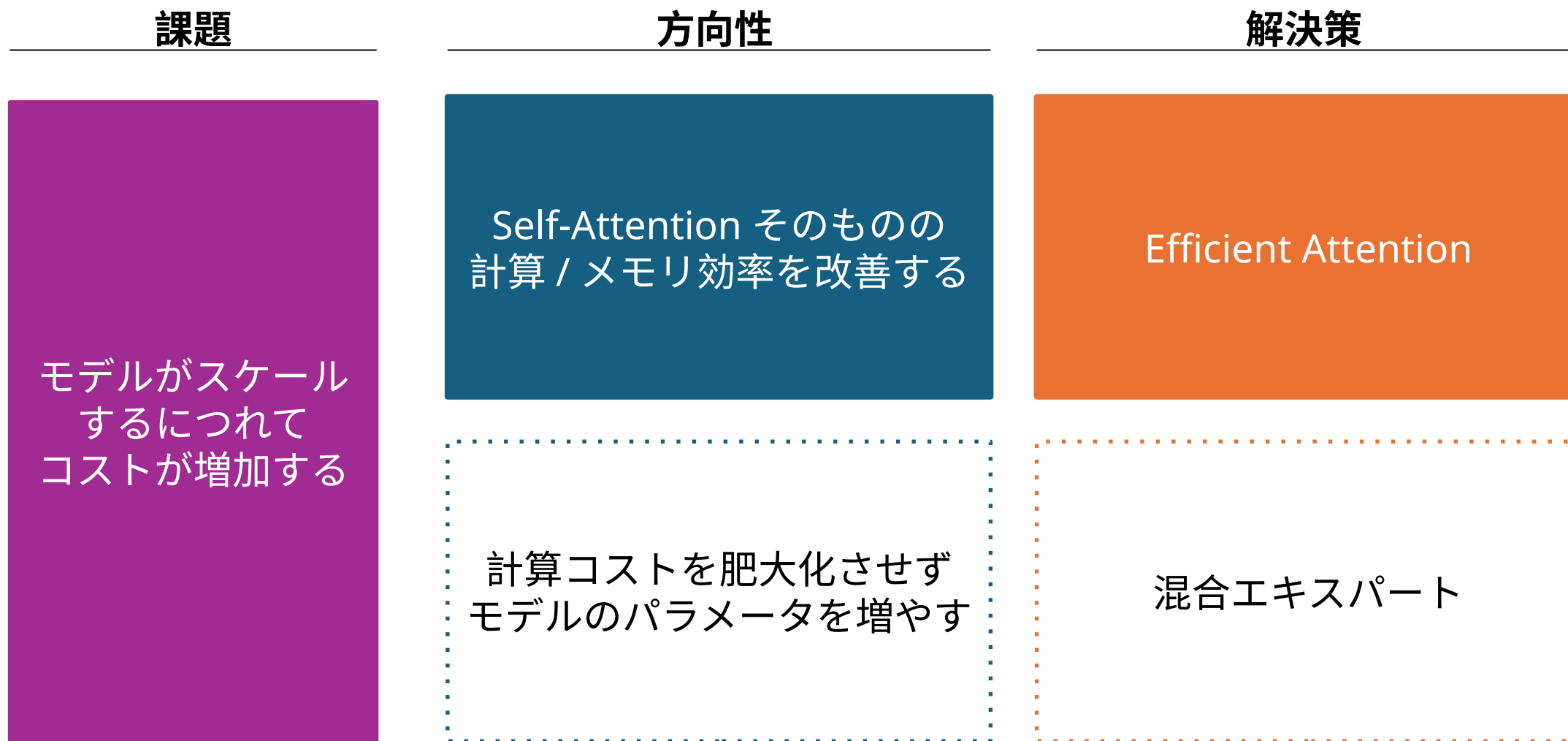
- それぞれの要素のスケールにおける問題点
- **スケールするための技術: パラメータ数 (N) に関連する取り組み**
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

パラメータ (N) に関連する取り組みの全体像

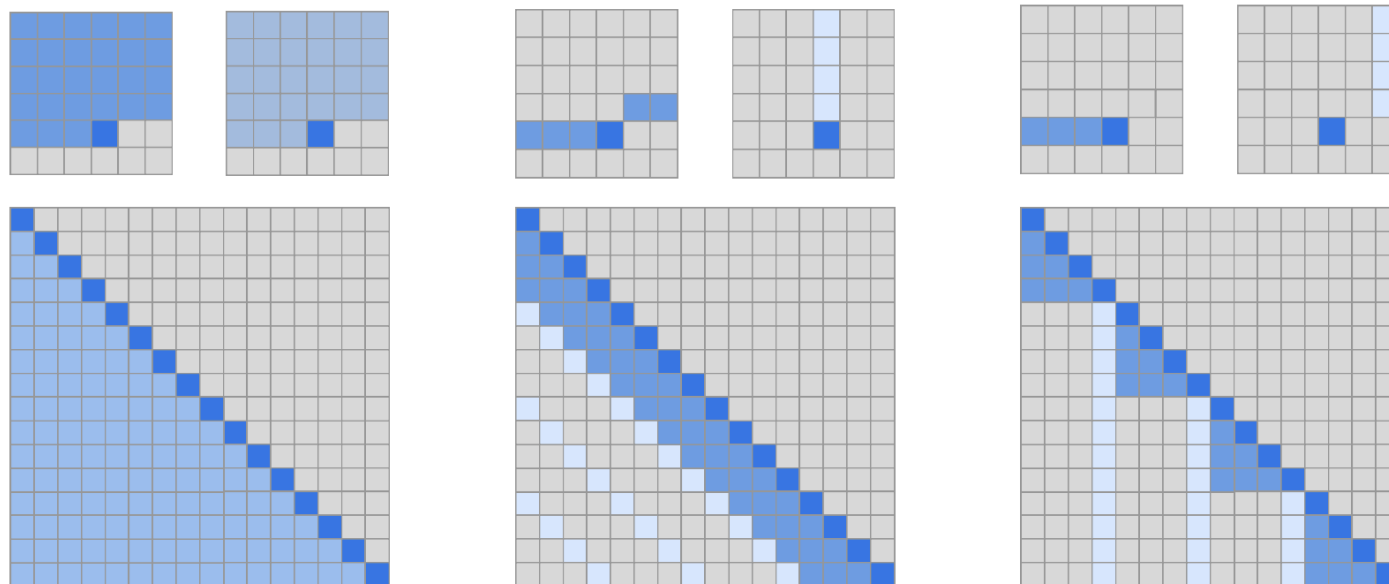


課題	方向性	解決策
モデルがスケールするにつれてコストが増加する	Self-Attention そのものの計算 / メモリ効率を改善する	Efficient Attention
	計算コストを肥大化させずモデルのパラメータを増やす	混合エキスパート

パラメータ (N) に関連する取り組みの全体像



Sparse Transformer : Sparse(疎)な Attention の提案



(a) Transformer

(b) Sparse Transformer (strided)

(c) Sparse Transformer (fixed)

Model	Bits per byte
CIFAR-10	
PixelCNN (Oord et al., 2016)	3.03
PixelCNN++ (Salimans et al., 2017)	2.92
Image Transformer (Parmar et al., 2018)	2.90
PixelSNAIL (Chen et al., 2017)	2.85
Sparse Transformer 59M (strided)	2.80
Enwik8	
Deeper Self-Attention (Al-Rfou et al., 2018)	1.06
Transformer-XL 88M (Dai et al., 2018)	1.03
Transformer-XL 277M (Dai et al., 2018)	0.99
Sparse Transformer 95M (fixed)	0.99
ImageNet 64x64	
PixelCNN (Oord et al., 2016)	3.57
Parallel Multiscale (Reed et al., 2017)	3.7
Glow (Kingma & Dhariwal, 2018)	3.81
SPN 150M (Menick & Kalchbrenner, 2018)	3.52
Sparse Transformer 152M (strided)	3.44
Classical music, 5 seconds at 12 kHz	
Sparse Transformer 152M (strided)	1.97

[19] Child+. Generating Long Sequences with Sparse Transformers. 2019 より引用

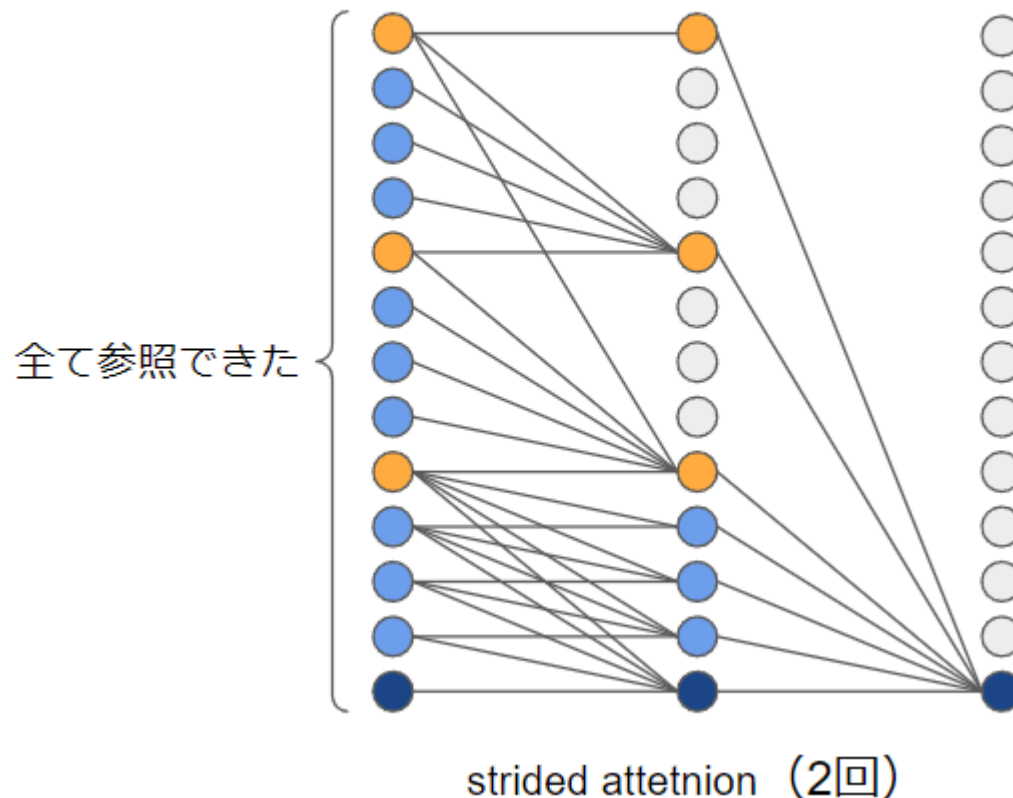
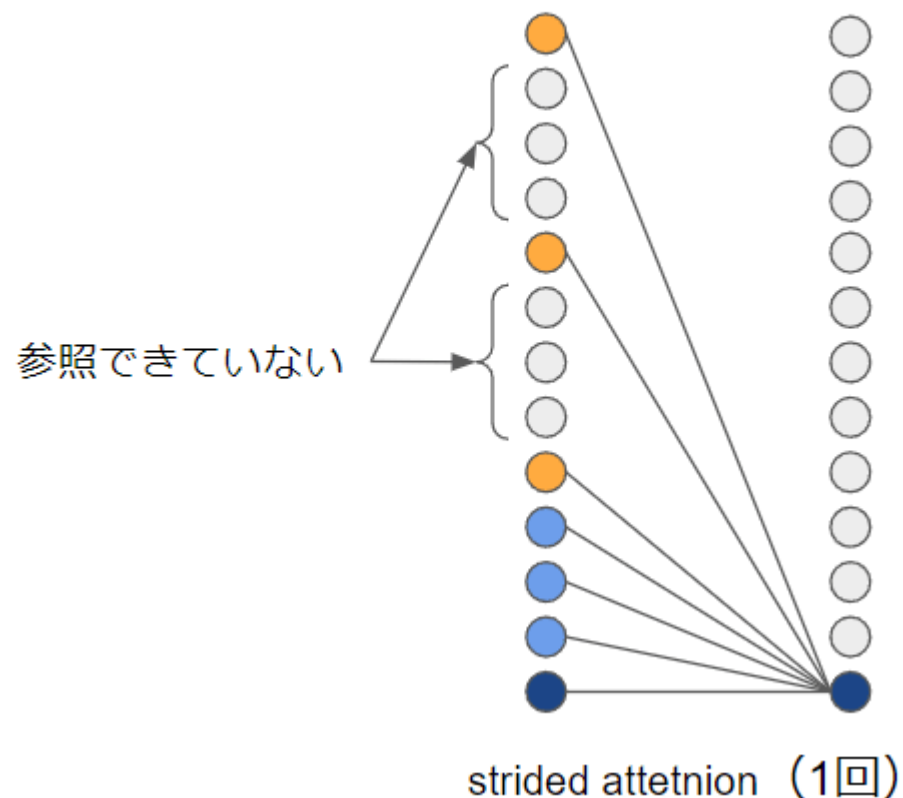
- Attention を計算する箇所を限定 (計算しない箇所はマスク) することで計算量削減
- 非常に長い系列長の入力 (例 : 画像や音声) に対しても効率的に Transformer の利用が可能に .

Sparse Transformer : Sparse(疎)な Attention の提案



●: Attention head 1の参照範囲 ●: Attention head 2の参照範囲 ●: 出力

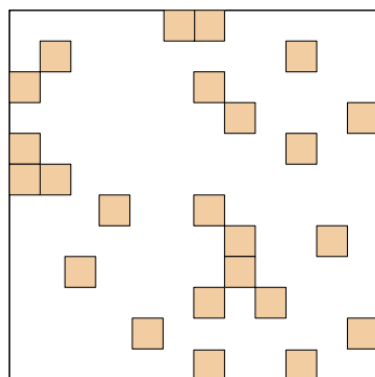
2回アテンション機構を
通せば全てのトークンに
アテンションが当たる。



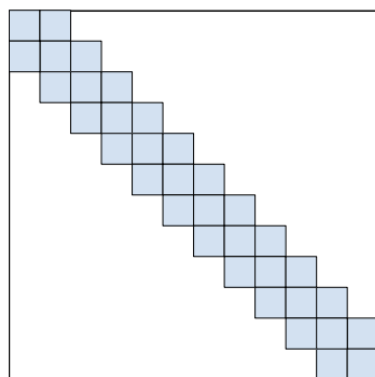
[19] Child+. Generating Long Sequences with Sparse Transformers. 2019 より引用

[64] sunbluesome. Sparse Transformer を理解したいより引用

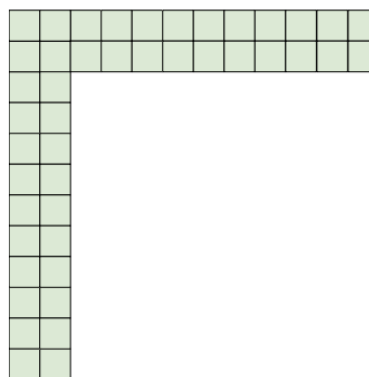
Big Bird : Sparse(疎)な Attention の提案



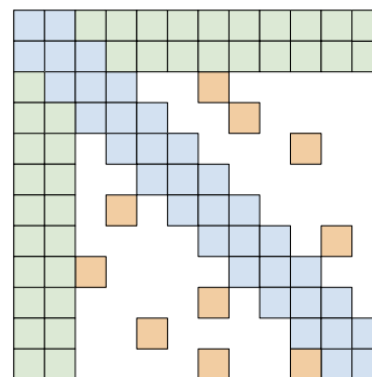
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

方法
複数の Sparse な Attention を組み合わせて、Attention を近似し、長い系列に対応する

Model	HotpotQA			NaturalQ		TriviaQA		WikiHop
	Ans	Sup	Joint	LA	SA	Full	Verified	MCQ
HGN [26]	82.2	88.5	74.2	-	-	-	-	-
GSAN	81.6	88.7	73.9	-	-	-	-	-
ReflectionNet [32]	-	-	-	77.1	64.1	-	-	-
RikiNet-v2 [61]	-	-	-	76.1	61.3	-	-	-
Fusion-in-Decoder [39]	-	-	-	-	-	84.4	90.3	-
SpanBERT [42]	-	-	-	-	-	79.1	86.6	-
MRC-GCN [87]	-	-	-	-	-	-	-	78.3
MultiHop [14]	-	-	-	-	-	-	-	76.5
Longformer [8]	81.2	88.3	73.2	-	-	77.3	85.3	81.9
BIGBIRD-ETC	81.2	89.1	73.6	77.8	57.9	84.5	92.4	82.3

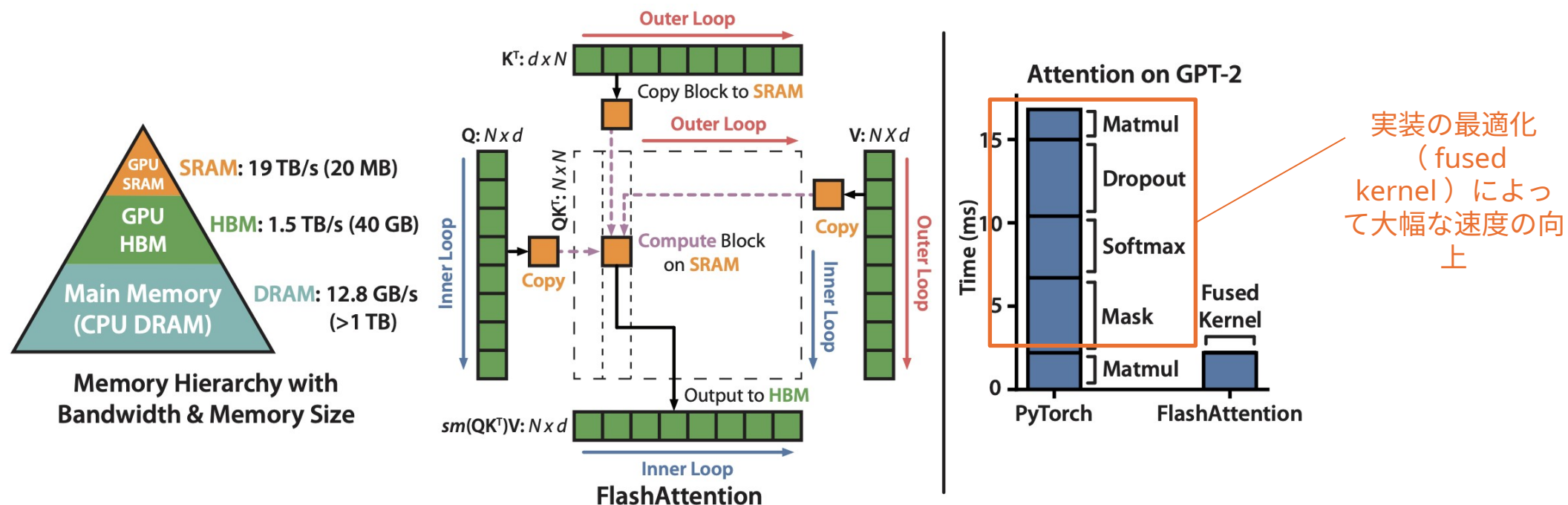
結果
長い系列を扱う質問応答や要約などのタスクで良い性能を獲得

[20] Zahher+. Big Bird: Transformers for Longer Sequences. 2020 より引用

類似アイデア： ” Longformer: The Long-Document Transformer”, 2020

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

FlashAttention : メモリアクセスを考慮し高速化を実現



[21] Dao+, FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022 より引用

Attention の計算は、計算ではなくメモリの I/O にボトルネックがあることを指摘

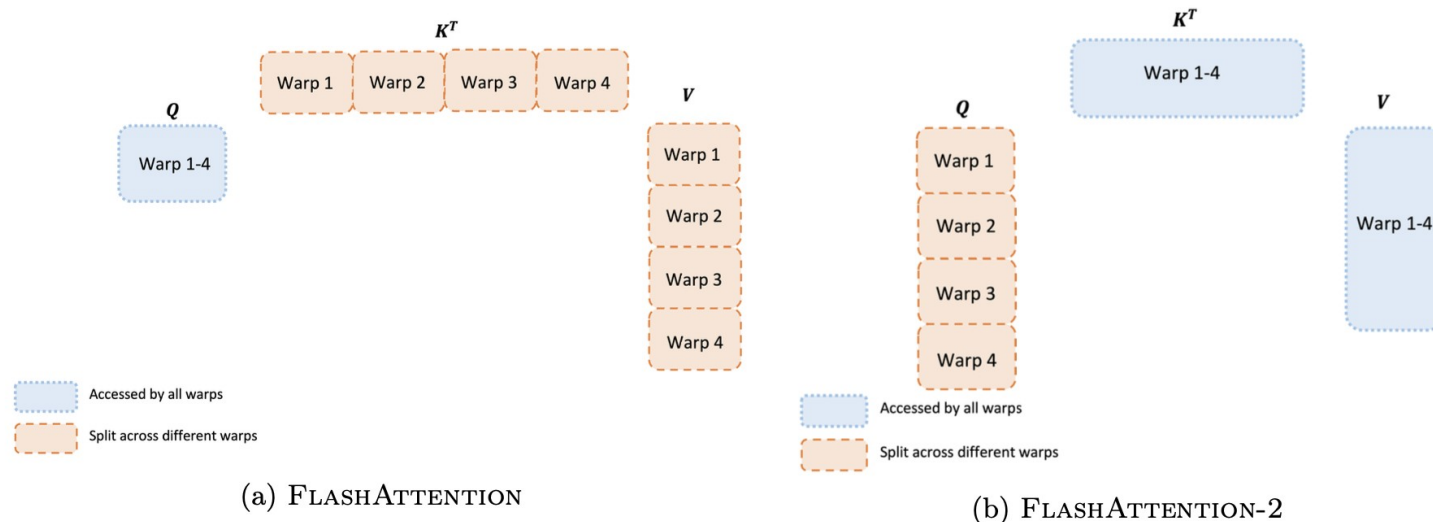
入力行列をうまく分割して計算することで、系列長 × 系列長の Attention 行列全体のメモリの読み書きを回避。GPU SRAM で処理をなるべく完結させる

(低速な GPU HBM メモリへのアクセス回数を削減) ようにし、大幅な高速化 (e.g. GPT-2 において最大 7.6 倍) に成功

FlashAttention2 : FlashAttention をさらに高速化



“FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning” [22]



方法：実装上の3つの工夫を積み上げることによる高速化

[22] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023 より引用

- アルゴリズムを工夫することで行列演算以外の演算をなるべく削減する
(GPUは行列演算は専用の計算ユニットがあるため高速に処理できる)
- バッチやAttentionのヘッドだけでなく系列方向にも並列演算を実施することでバッチやヘッド数が少ない場合でも高速化できるようにする
- ワープ（同時に走るスレッドのグループを指すGPUの用語）をクエリ行列で分割することで、ワープ間の同期や通信を削減し、並列性を向上

Flash-Attention のおよそ2倍の高速化 (PyTorch の標準 Attention の最大9倍高速)

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

4.0



計算コストを肥大化させずモデルのパラメータを増やす？



用意する
パラメータ数



通常

必要な計算量



用意する
パラメータ数



必要な計算量



用意する
パラメータ数



必要な計算量

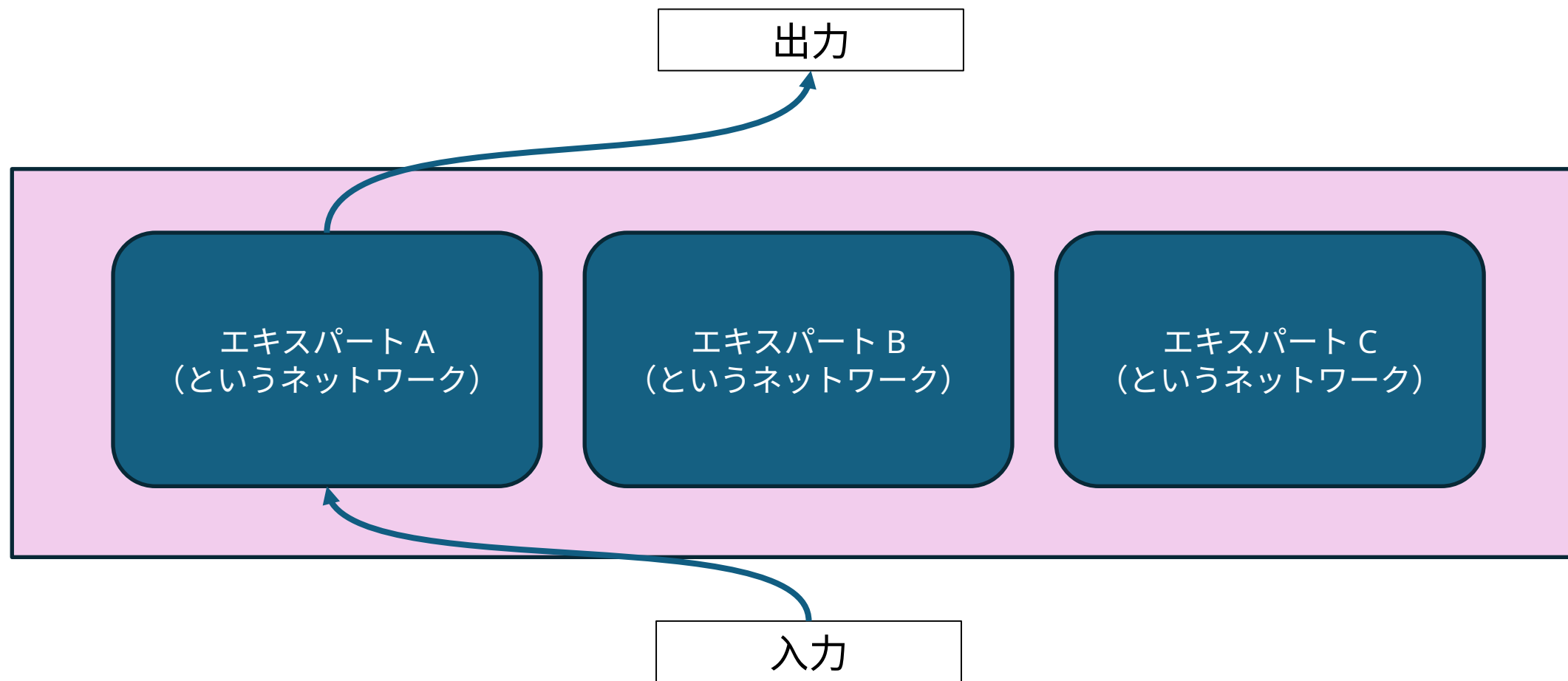


やりたい事

混合エキスパート (MoE)



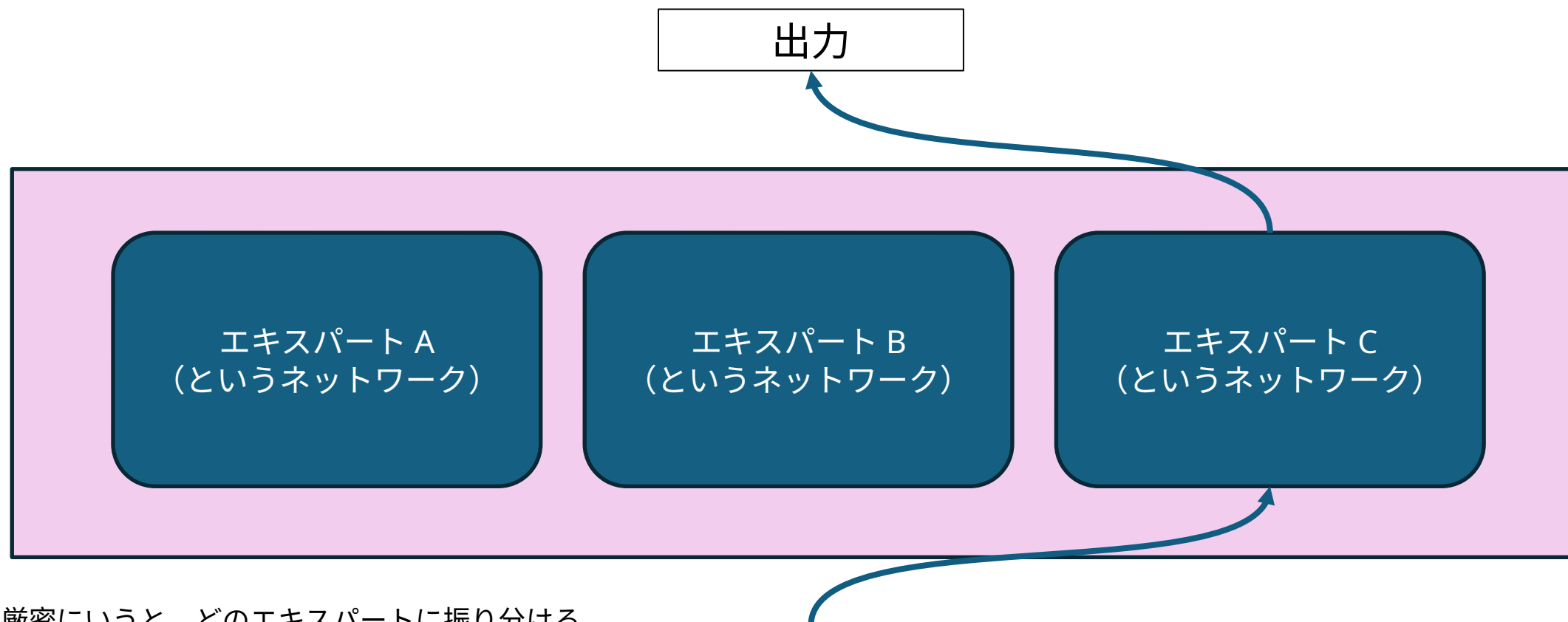
複数のエキスパート (ニューラルネットワーク) を用意しておき, 入力の値に応じて, 一部のエキスパートだけにフォワードする. → すべてのパラメータを使う事にはならないので, 計算量を抑えられる.



混合エキスパート (MoE)



複数のエキスパート (ニューラルネットワーク) を用意しておき, 入力の値に応じて, 一部のエキスパートだけにフォワードする. → すべてのパラメータを使う事にはならないので, 計算量を抑えられる.



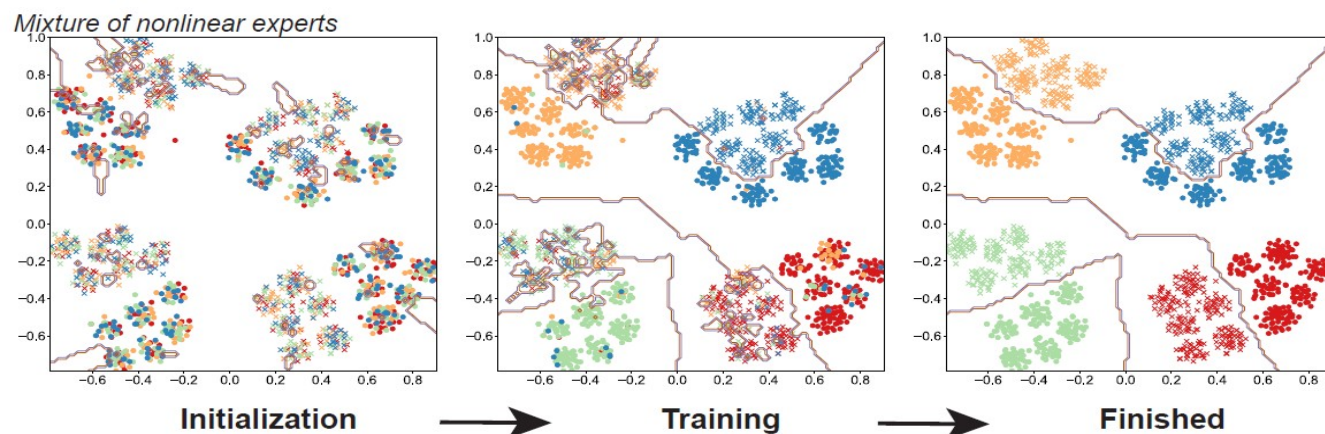
*厳密にいうと, どのエキスパートに振り分けるかを定めるための小さなネットワーク (ルーターネットワーク) が追加で必要となるため, その分若干計算量は増える.

混合エキスパート (MoE)

- 計算量を抑えつつ、パフォーマンスを改善できる事が実験で確認されている。
→ 同じ計算量で学習するという制約下で、MoE を使ったモデルのほうが使わなかったモデル
(通常のモデル) よりもパフォーマンスが高い。
- リーク情報によると、GPT-4 は MoE モデル構造を採用しているらしい。
- 最近の多くのオープンモデル (DeepSeek、Qwen 等) も MoE モデルを採用

複雑な分類問題に MoE を適用した場合の各事例に対するルータネットワークによるエキスパートの振り分けの視覚化

(t-SNE でデータセットの事例を 2 次元で視覚化。色はエキスパートの振り分けを示す)



ルータネットワークが
データのクラスター中心点を
基準に各エキスパートに対して
事例を振り分け、
各エキスパートはその
クラスター内での分類に特化する
ような学習が行われた

[23] Chen+. Towards Understanding Mixture of Experts in Deep Learning. 2022 より

引用

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

4.0

Switch Transformer : 1兆6000億パラメータのMoEモデル



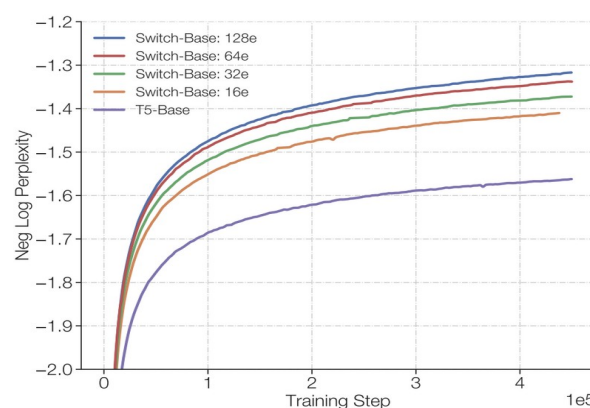
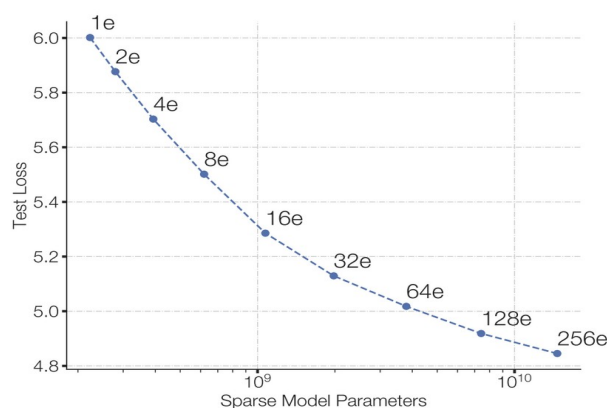
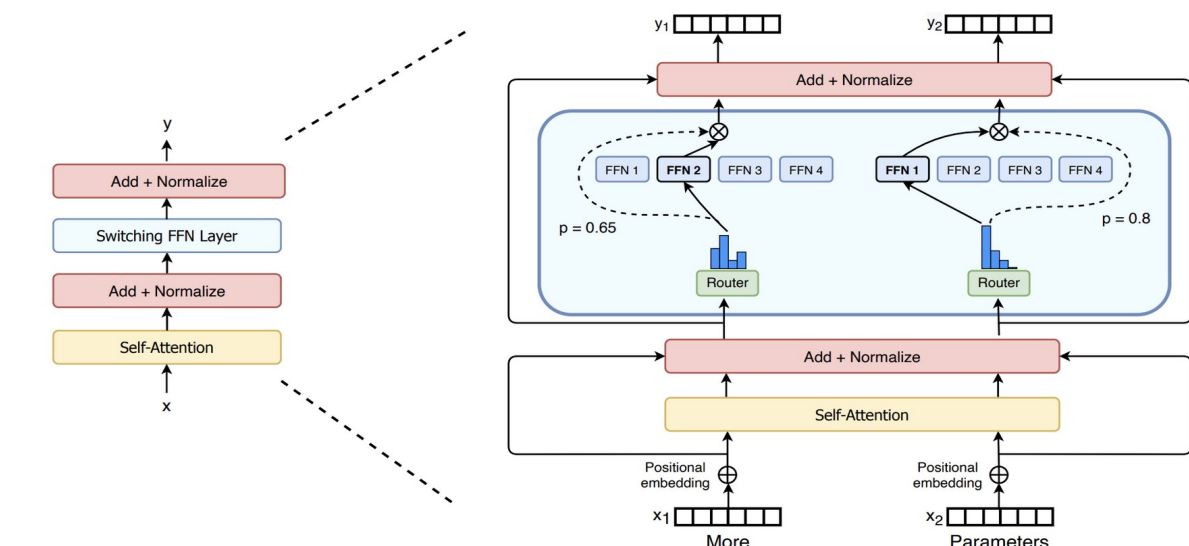
方法

T5モデルのフィードフォワード層にMoEを適用して、大規模化

多くのMoEでは各トークンごとに複数のエキスパートが使われるがこれを一つのエキスパートのみを使うようにすることで、通信・計算コストの削減を実現

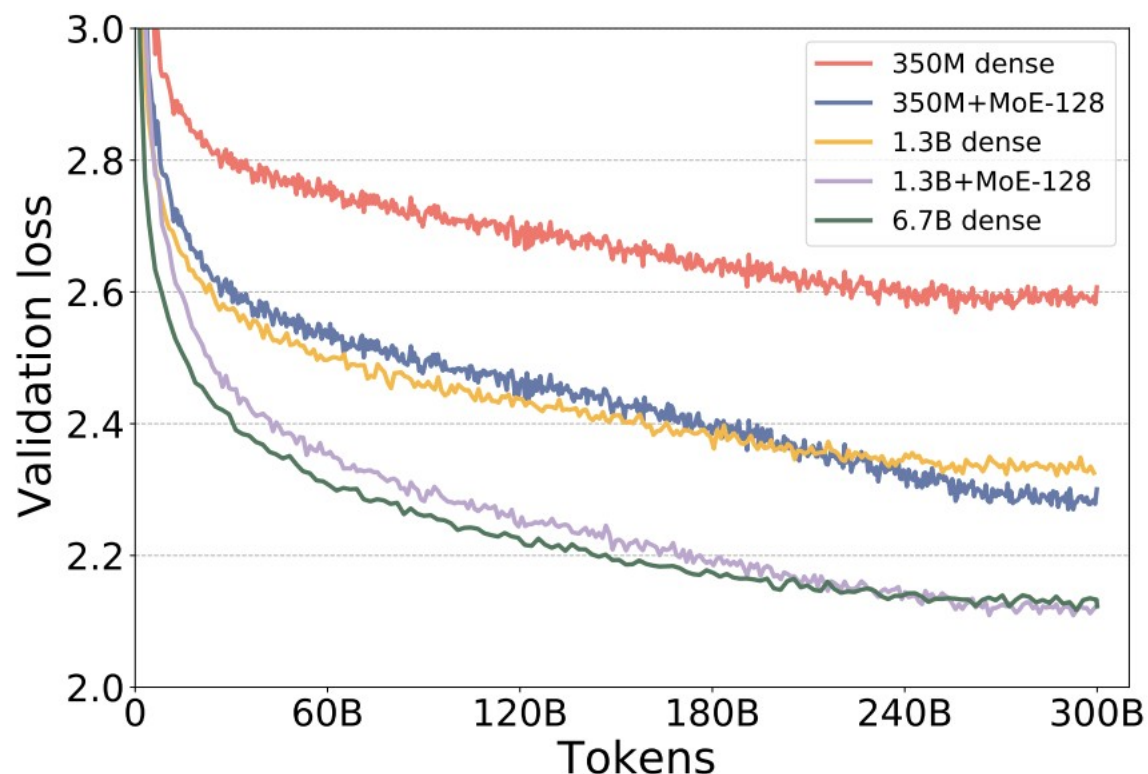
結果

1.6兆パラメータのモデルの学習でT5-XXLモデルに対して4倍の事前学習のスピードアップ



[25] Fedus+. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. 2021 より引用

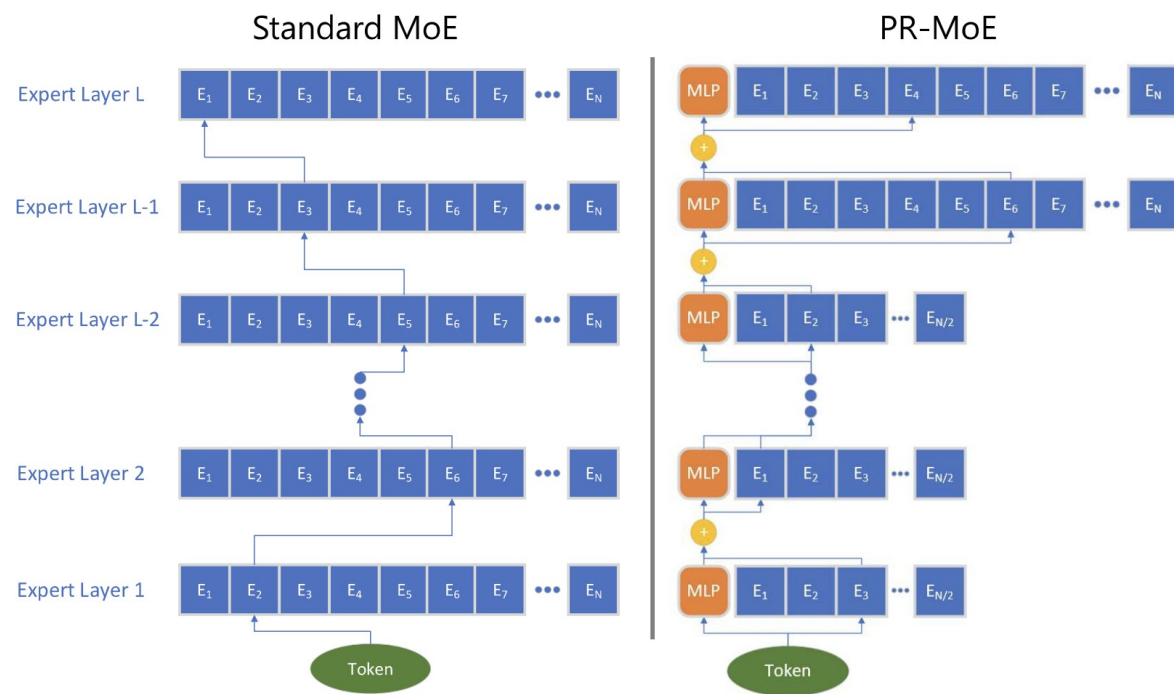
LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND



DeepSpeed MoE という最適化された実装により，自己回帰モデルでの品質が同等の Dense モデルと比較して 5 倍程度学習コストを削減した MoE の学習を実現

[26] Rajbhandari+. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. 2022 より引用

PR-MoE： MoE モデルの性能を維持したモデルサイズの削減



方法

PR-MoE というアーキテクチャを提案

- 各トークンが **1つの固定された MLP** と **1つのエキスパート** の双方を利用
- Transformer の **後半の層において** **より多くのエキスパートを活用する**

結果

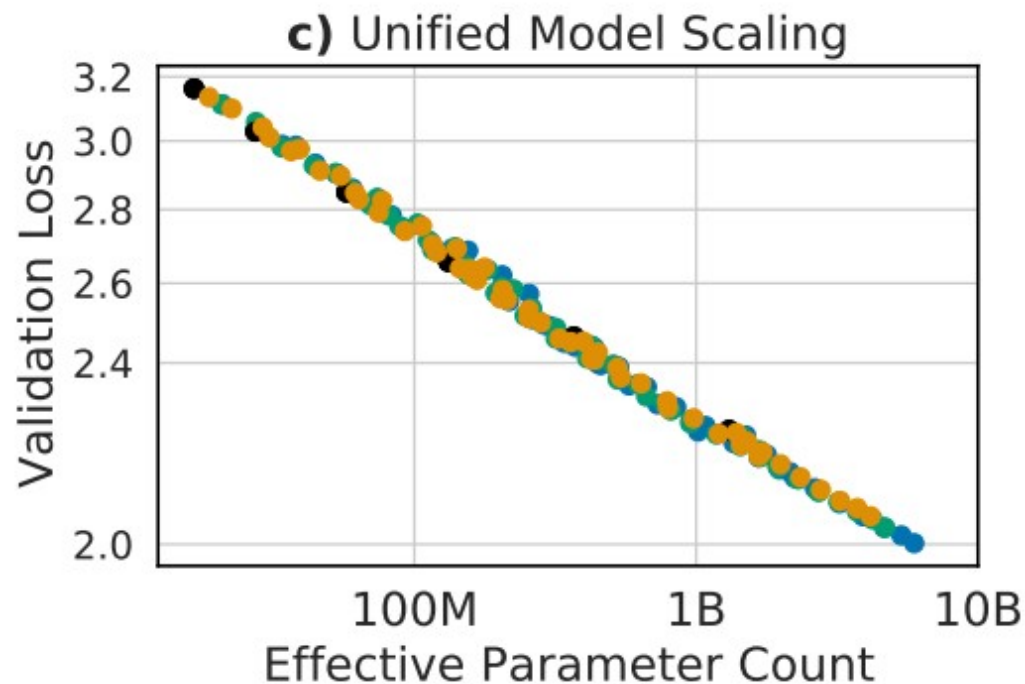
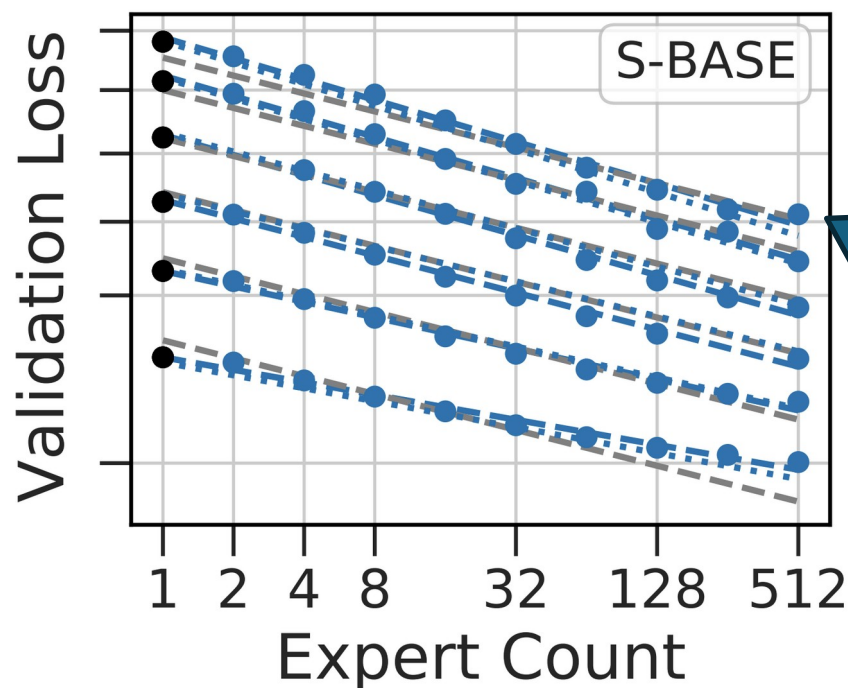
標準的な MoE より少ないパラメータ数で同等の性能を達成

- 350M: 1/3 以下のパラメータで同等性能
- 1.3B: Standard-MoE の約 60% のパラメータで同等性能

Model (num. params)	LAMBADA	PIQA	BoolQ	RACE-h	TriviaQA	WebQs
350M+MoE-128 (13B)	62.70	74.59	60.46	35.60	16.58	5.17
350M+PR-MoE-32/64 (4B)	63.65	73.99	59.88	35.69	16.30	4.73
1.3B+MoE-128 (52B)	69.84	76.71	64.92	38.09	31.29	7.19
1.3B+PR-MoE-64/128 (31B)	70.60	77.75	67.16	38.09	28.86	7.73

[26] Rajbhandari+. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. 2022 より引用

MoE を含む言語モデルにおけるスケール則



[27] Clark+. Unified Scaling Laws for Routed Language Models. 2022 より引用

(左図) エキスパート数を増やすと対数損失が下がっていくが、特に大きいモデルサイズで、増やしすぎると効果が薄くなる (灰色の線が線形にフィットさせたもの)

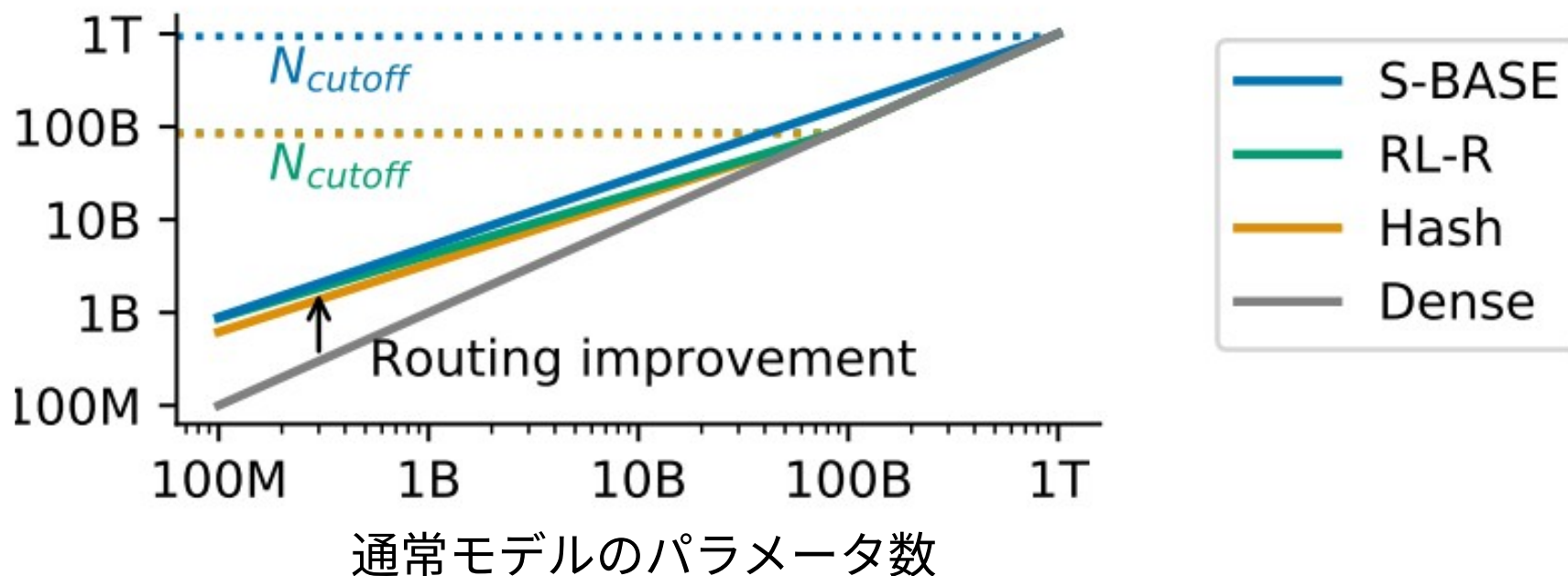
(右図) MoE モデルのパラメータ数を諸々の要素を加味して通常のモデルのパラメータ数に換算すると、スケール則が成立

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

MoE を含む言語モデルにおけるスケール則



通常モデルを MoE 化した際に
効果のある最大パラメータ数



[27] Clark+. Unified Scaling Laws for Routed Language Models. 2022 より引用

通常モデルのパラメータ数が大きくなると
MoE 化する効果は比例して低くなっていく



モデルサイズに適したエキスパート数を選択すると良い

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

4.0

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- **スケールするための技術: 計算量 (C) に関連する取り組み**
- スケールするための技術: データ (D) に関連する取り組み

計算量 (C) に関する取り組みの全体像

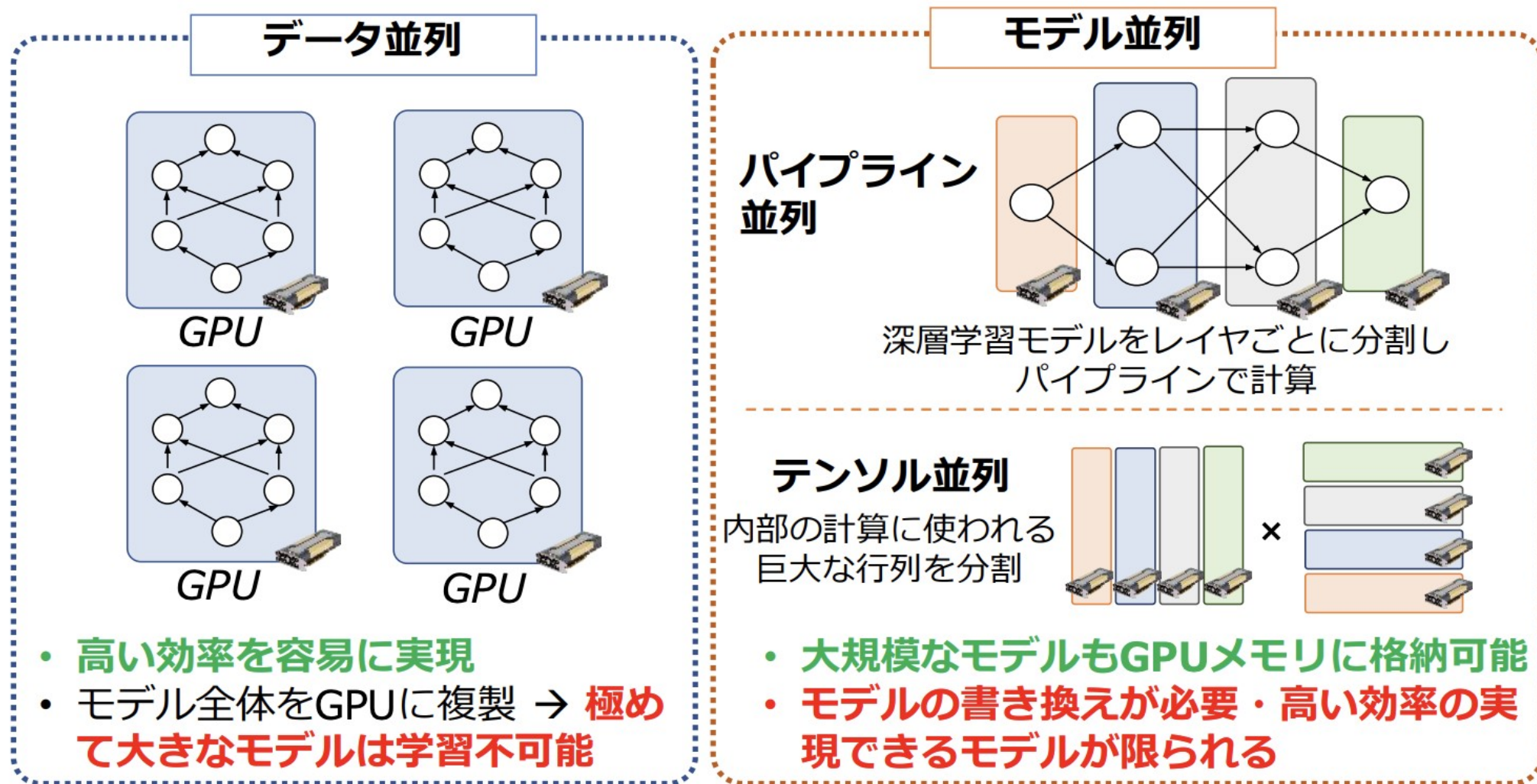


課題	方向性	解決策
十分な計算量 / メモリ量を確保し 効率よく 訓練する必要	訓練において複数の GPU を 効率的に活用する	並列計算
	(主に推論時) モデルの軽量 化を通じて、小規模な GPU 環境での運用を可能にする	量子化

課題	方向性	解決策
十分な計算量 / メモリ量を確保し 効率よく 訓練する必要	訓練において複数の GPU を 効率的に活用する	並列計算
	(主に推論時) モデルの軽量 化を通じて、小規模な GPU 環境での運用を可能にする	量子化

深層学習における並列化

“DeepSpeed: 深層学習の訓練と推論を劇的に高速化するフレームワーク” [32] より抜粋,



[32] Microsoft DeepSpeed Team, DeepSpeed: Extreme-scale model training for everyone, Microsoft より

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

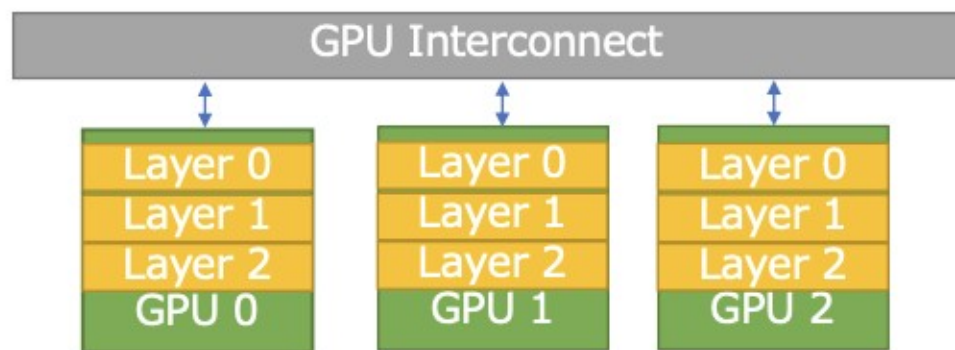
ZeRO : データ並列時のメモリ効率化



“DeepSpeed: 深層学習の訓練と推論を劇的に高速化するフレームワーク” [32] より抜粋,

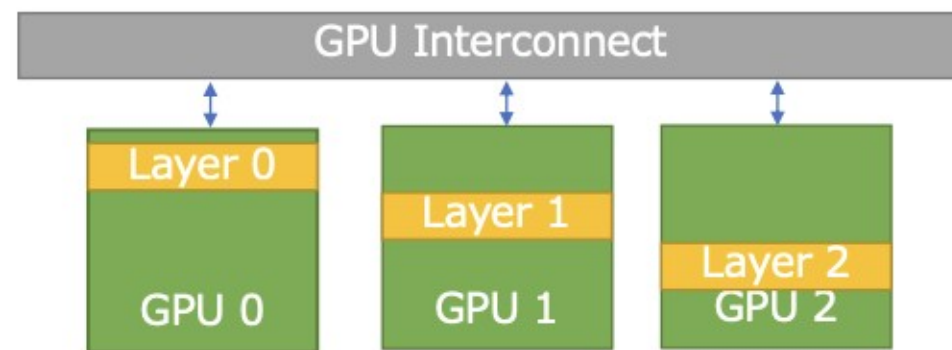
- データ並列のメモリ利用を効率化
→ 巨大モデルを高い効率で学習、かつどのようなモデルアーキテクチャでも適用可能

通常データ並列



Model States mapping in **Data Parallel** Training

ZeRO (**Z**ero **R**edundancy **O**ptimizer)



Model States mapping in **ZeRO** Training

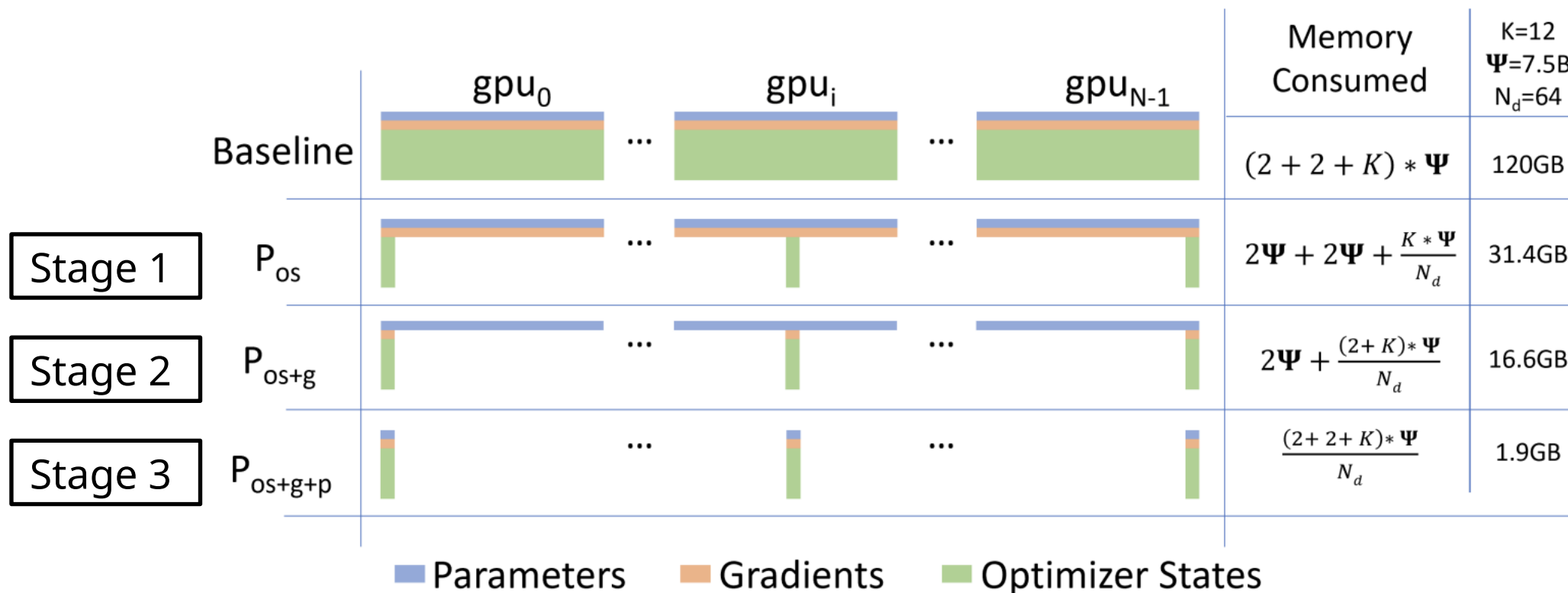
[32] Microsoft DeepSpeed Team, DeepSpeed: Extreme-scale model training for everyone, Microsoft より引用

- 通常データ並列では重複して持つ学習パラメータを、重複のないように各GPUに格納
- データが必要になった際に、GPU間の通信で必要な部分だけデータを収集し、使い終わったら捨てる

ZeRO : データ並列時のメモリ効率化



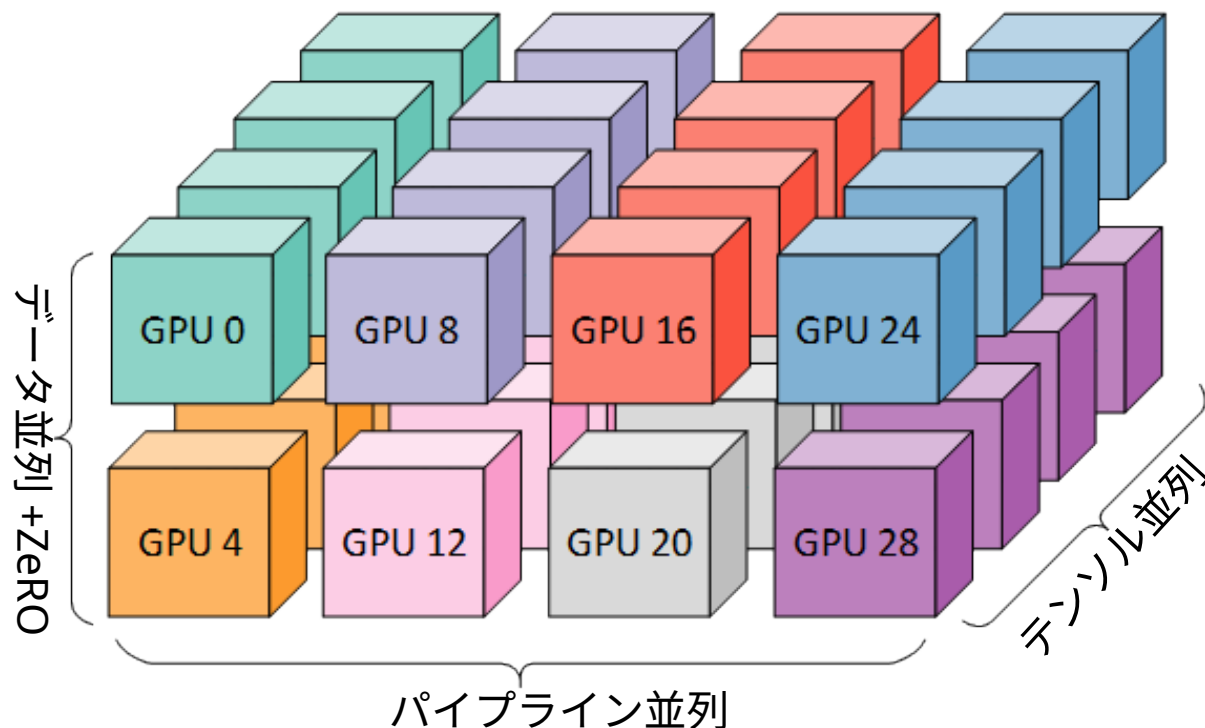
- どの要素をメモリで並列するかに応じて，3 段階の動作モードが存在
- 段階が進めほどメモリを削減できるが，通信オーバーヘッドが増加する



[33] Rajbhandari+. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. 2019 より引用

“DeepSpeed: 深層学習の訓練と推論を劇的に高速化するフレームワーク” [32] より抜粋,

- 並列化の戦略ごとに通信オーバーヘッドが異なる
テンソル並列 >> パイプライン並列
- 3D 並列化: GPU・ノードの配置に応じて通信コストをおさえて並列化



**4 枚の GPU を持つ 8 ノードで
3D parallelism を構成した例**

同色の GPU は同一ノードに配置されていることをあらわす

- 高オーバーヘッドのテンソル並列をノード内に配置
- 低オーバーヘッドのパイプライン並列をノードをまたいで配置
- データ並列と ZeRO stage 1 の組み合わせによって、GPU メモリの効率を高める

[32] Microsoft DeepSpeed Team, DeepSpeed: Extreme-scale model training for everyone, Microsoft より

引用

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

■ 補足 | ZeRO は環境設定を記述するだけで利用可能



*代表的なライブラリ: deepspeed

```
"zero_optimization": {
  "stage": 2,
  "offload_optimizer": {
    "device": "cpu",
    "pin_memory": true
  },
  "allgather_partitions": true,
  "allgather_bucket_size": 2e8,
  "overlap_comm": true,
  "reduce_scatter": true,
  "reduce_bucket_size": 2e8,
  "contiguous_gradients": true
},
```

```
"zero_optimization": {
  "stage": 3,
  "offload_optimizer": {
    "device": "cpu",
    "pin_memory": true
  },
  "offload_param": {
    "device": "cpu",
    "pin_memory": true
  },
  "overlap_comm": true,
  "contiguous_gradients": true,
  "reduce_bucket_size": "auto",
  "stage3_prefetch_bucket_size": "auto",
  "stage3_param_persistence_threshold": "auto",
  "sub_group_size": 1e9,
  "stage3_max_live_parameters": 1e9,
  "stage3_max_reuse_distance": 1e9,
  "stage3_gather_16bit_weights_on_model_save": "auto"
}
```

[34] Microsoft, <https://github.com/microsoft/DeepSpeed>より引用

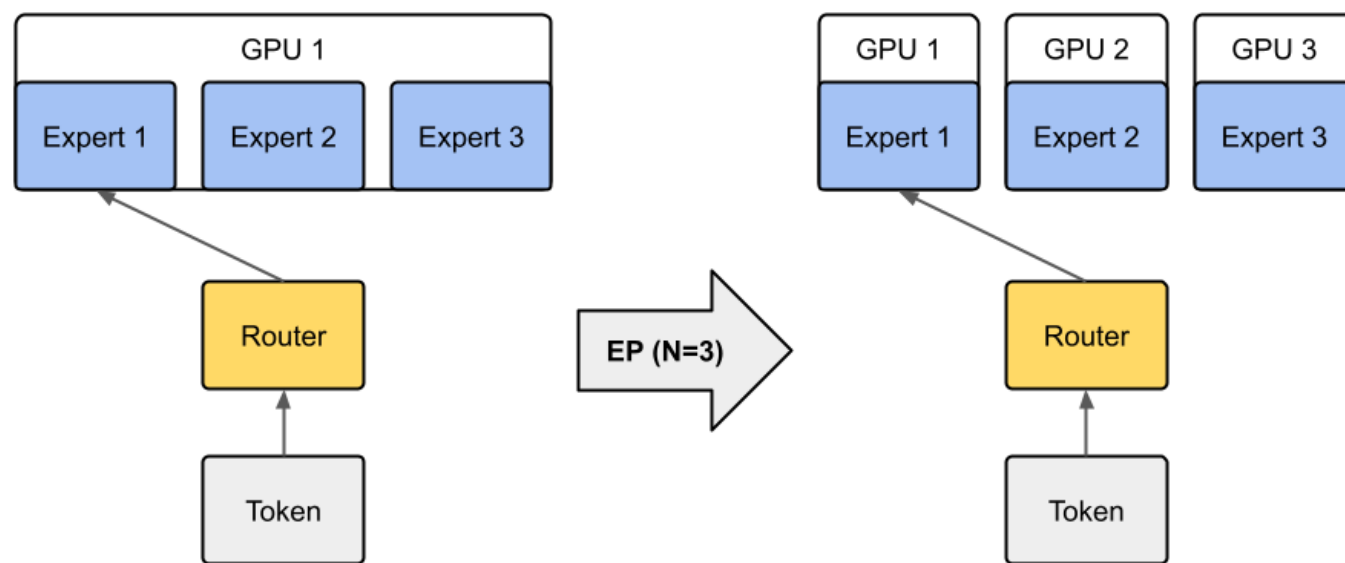
[35] DeepSpeed, <https://www.deepspeed.ai/docs/config-json/>より引用

エキスパート並列化



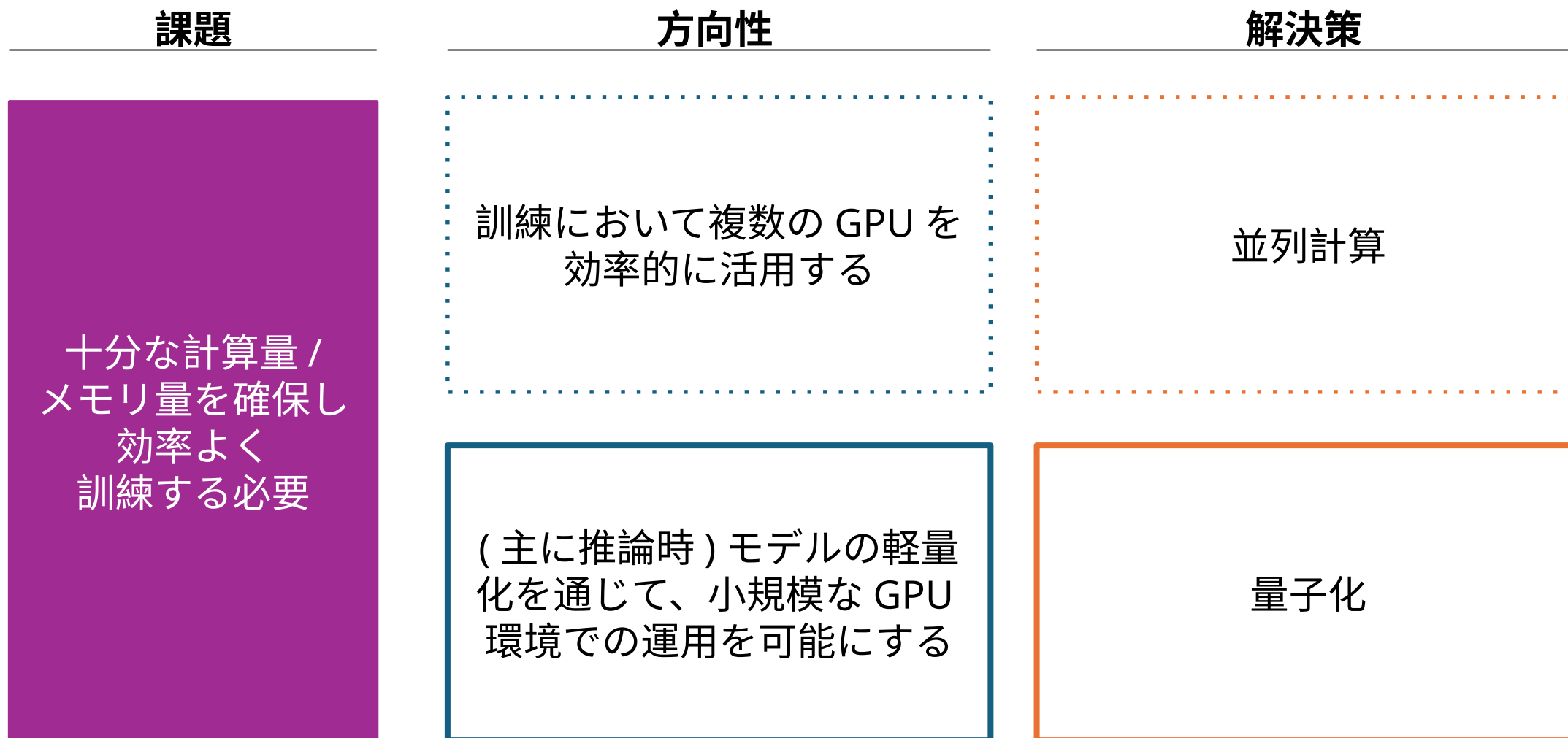
“NVIDIA NeMo Framework User Guide - Parallelisms”^[55] より抜粋，

- MoE モデル専用の並列化手法
- MoE の各エキスパートを異なる GPU に配置する
- 行列を分割して複数の GPU で保持するテンソル並列と類似している
- 全ての層に適用されるテンソル並列化と異なり、**エキスパート並列化は、エキスパート層にのみ適用**される

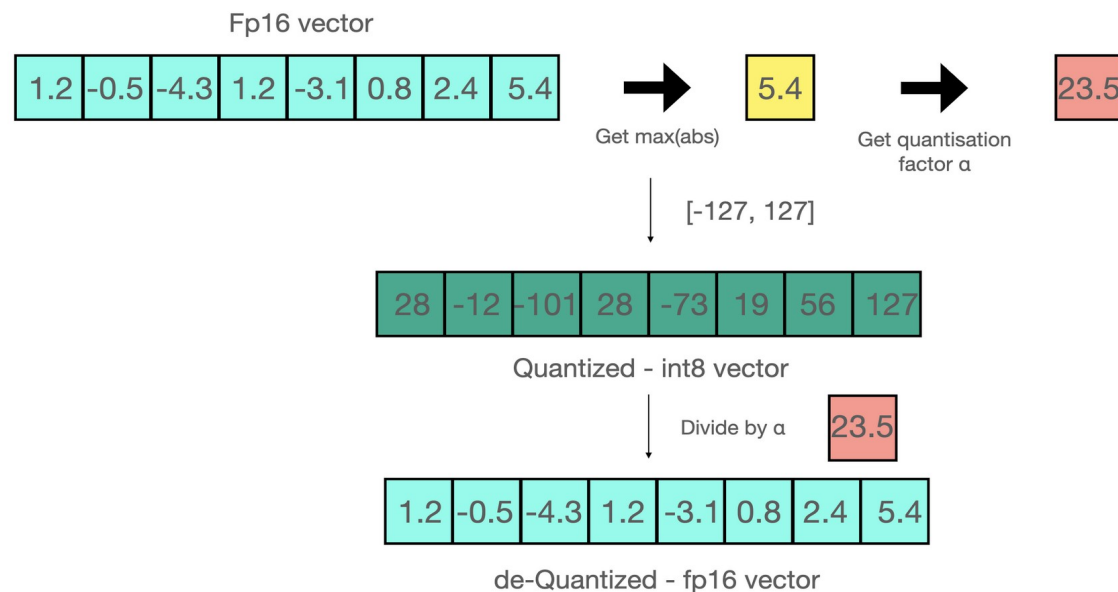


[55] NVIDIA NeMo Framework User Guide - Parallelisms, NVIDIA より引用

計算量 (C) に関する取り組み: 量子化

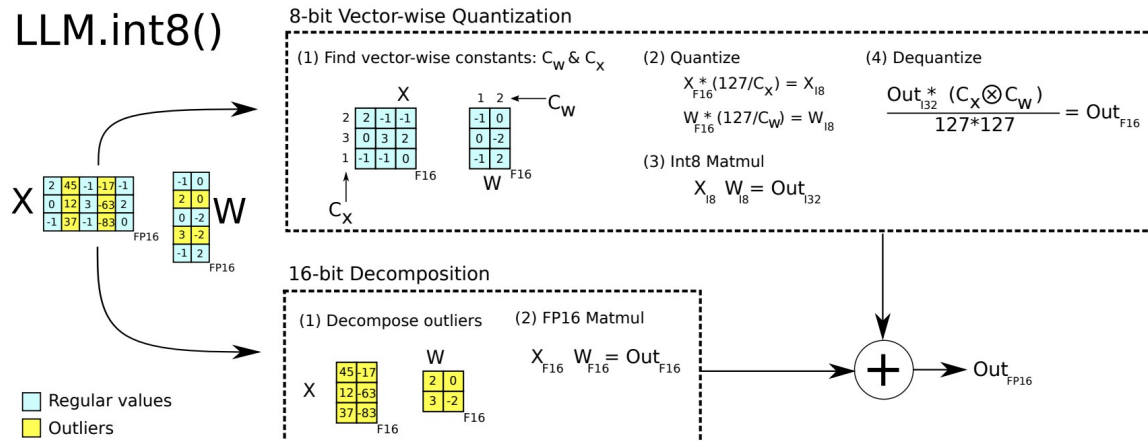


- モデルパラメータのデータタイプを
浮動小数点 (Float 型) から整数 (Int 型) に変換して演算処理を行う
- 推論時に, 必要メモリ量が削減できる.
- ナイーブにこれを行うと性能劣化が発生する.



[63] Younes Belkada, Tim Dettmers, A Gentle Introduction to 8-bit Matrix Multiplication for transformers at scale using Hugging Face Transformers, Accelerate and bitsandbytes, Hugging Face, <https://huggingface.co/blog/hf-bitsandbytes-integration#a-gentle-summary-of-llm-int8-zero-degradation-matrix-multiplication-for-large-language-models> より引用

LLM.int8()：性能劣化なしに可能な量子化方法



方法

16 ビットの**行列乗算**で異常値の特徴を分離する混合精度分解を行い、
大部分の値を 8 ビットで、
外れ値のみを 16 ビットで表現する

結果

16 ビットと比較して約 50% のメモリ削減が可能な場合を示す

175B までのパラメータを持つ LLM において、性能劣化なしに推論を行うことが可能であることを経験的に示す

Table 1: C4 validation perplexities of quantization methods for different transformer sizes ranging from 125M to 13B parameters. We see that absmax, row-wise, zeropoint, and vector-wise quantization leads to significant performance degradation as we scale, particularly at the 13B mark where 8-bit 13B perplexity is worse than 8-bit 6.7B perplexity. If we use LLM.int8(), we recover full perplexity as we scale. Zeropoint quantization shows an advantage due to asymmetric quantization but is no longer advantageous when used with mixed-precision decomposition.

Parameters	125M	1.3B	2.7B	6.7B	13B
32-bit Float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Int8 absmax row-wise + decomposition	30.76	16.19	14.65	13.25	12.46
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	13.24	12.45
Zeropoint LLM.int8() (vector-wise + decomp)	25.69	15.92	14.43	13.24	12.45

[37] Dettmers+. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. 2022 より引用

LLM.int8()：性能劣化なしに可能な量子化方法



Step1. 入力された隠れ状態から、列単位で外れ値（閾値より大きい値）を抽出する。

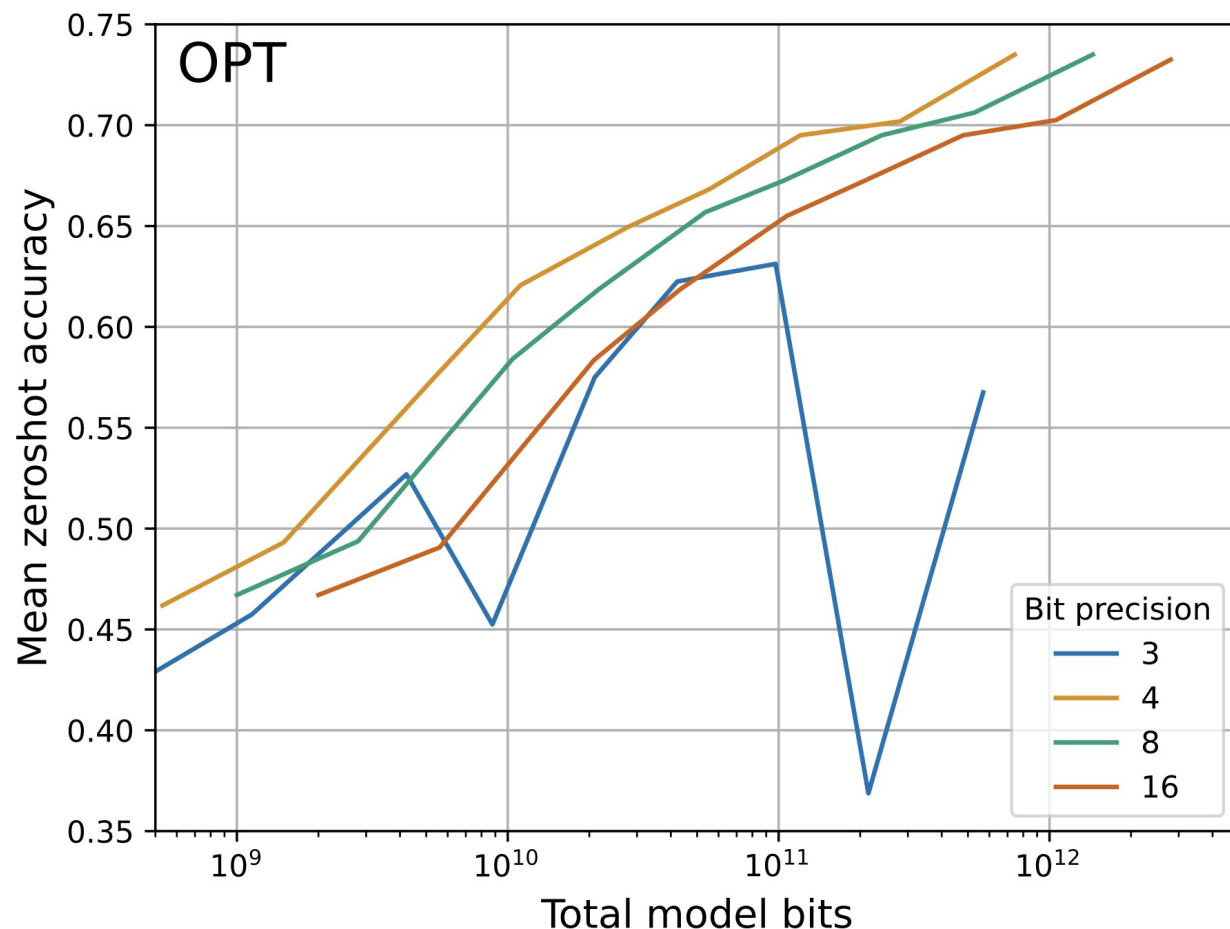
Step2. 外れ値の行列については、FP16 のまま行列演算．外れ値ではない行列については、INT8 に変換して（量子化して）行列演算．

Step3. 2つの出力値が存在する。INT8 の出力値は FP16 に戻して、2つの出力値を加算して、FP16 として出力値をリターンする．



[37] Dettmers+. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. 2022 より引用

[63] Younes Belkada, Tim Dettmers, A Gentle Introduction to 8-bit Matrix Multiplication for transformers at scale using Hugging Face Transformers, Accelerate and bitsandbytes, Hugging Face より引用



[56] Dettmers+. The case for 4-bit precision: k-bit Inference Scaling Laws. 2023 より引用

- モデルのメモリ容量（ビット数）を固定した際、モデルのサイズと量子化をどのように設定すべきか
例： 30B の 8-bit モデルと 60B の 4-bit モデルは同一のメモリ容量となる
- メモリ容量を固定した場合、4bit 量子化が最もゼロショットが高かった
- 3bit においてはモデルサイズが大きくなると性能が不安定に
→ post-hoc な量子化では、4-bit は最低必要？

量子化により Emergent Ability は失われないか



Models	Precision	MMLU (Acc)		BBH (Acc)		GSM8k (Acc)	AutoEval	WikiText (PPL)	Mem. (GiB)	Tokens/s
		0-shot	5-shot	0-shot	3-shot					
LLaMA-7B	16-bit	29.2	35.2	17.3	31.0	13.1	1121/1134	5.7	13.9	33.032
	8-bit	28.4	33.7	17.2	31.3	13.5	1092/1335	5.7	7.9	30.833
	4-bit	31.0	34.2	18.8	30.8	12.2	1058/1330	5.8	4.8	31.317
	2-bit	2.3	3.8	0.4	2.7	0.0	607/1263	3937.9	3.2	33.266
LLaMA-13B	16-bit	41.4	47.0	20.9	36.6	16.4	1084/1335	5.1	26.6	24.968
	8-bit	40.5	46.3	21.1	37.2	16.5	1084/1336	5.1	14.8	17.754
	4-bit	39.0	45.9	19.8	36.6	15.6	1119/1321	5.2	8.6	18.139
	2-bit	4.9	14.8	4.2	18.1	0.0	635/1258	142.6	5.5	18.422
LLaMA-30B	16-bit	53.7	58.4	19.5	39.4	34.7	1142/1317	4.1	65.4	16.596
	8-bit	54.2	57.9	19.9	39.4	34.7	1116/1325	4.1	35.3	8.187
	4-bit	53.7	57.3	18.3	40.2	35.4	1120/1325	4.2	20.0	8.371
	2-bit	3.7	26.1	3.8	25.3	0.2	630/1198	25.1	12.2	8.649
LLaMA-65B	16-bit	-	-	-	-	-	-	-	-	-
	8-bit	-	-	-	-	-	-	-	-	-
	4-bit	57.1	63.0	21.9	42.1	48.5	1171/1319	3.9	38.2	4.793
	2-bit	9.0	22.6	1.0	24.0	0.8	658/1309	77.8	22.9	4.826

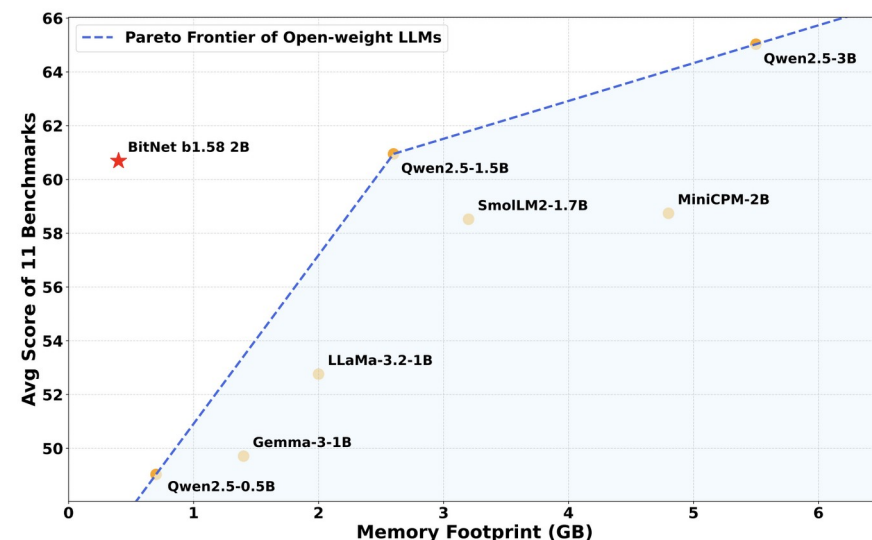
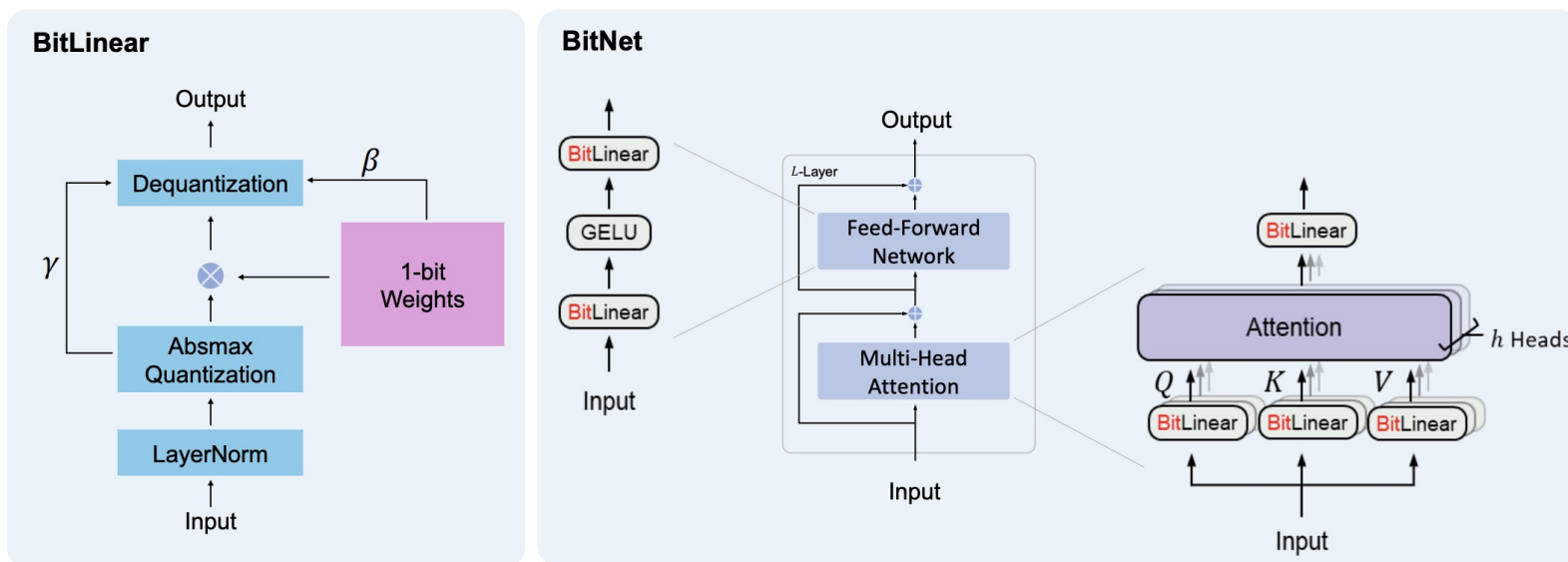
Table 1: Evaluation results on MMLU, BBH, GSM8k and AutoEval of the model variants in the LLaMA family. The results of the LLaMA-65B model at 16-bit and 8-bit precisions are not included due to memory constraints on a single GPU.

[38] Liu+. Do Emergent Abilities Exist in Quantized Large Language Models: An Empirical Study. 2023 より引用

- Emergent Ability は LLM の重要な特性
- in-context learning, chain-of-thought reasoning, instruction-following の能力を計測
- 結果として 4 ビットまでの量子化モデルでは Emergent Ability の維持を確認

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

訓練時から 1bit/1.58bit 量子化を行う : BitNet



[57] Wang+. BitNet: Scaling 1-bit Transformers for Large Language Models. 2023 より引用

[58] Ma+. BitNet b1.58 2B4T Technical Report. 2025 より引用

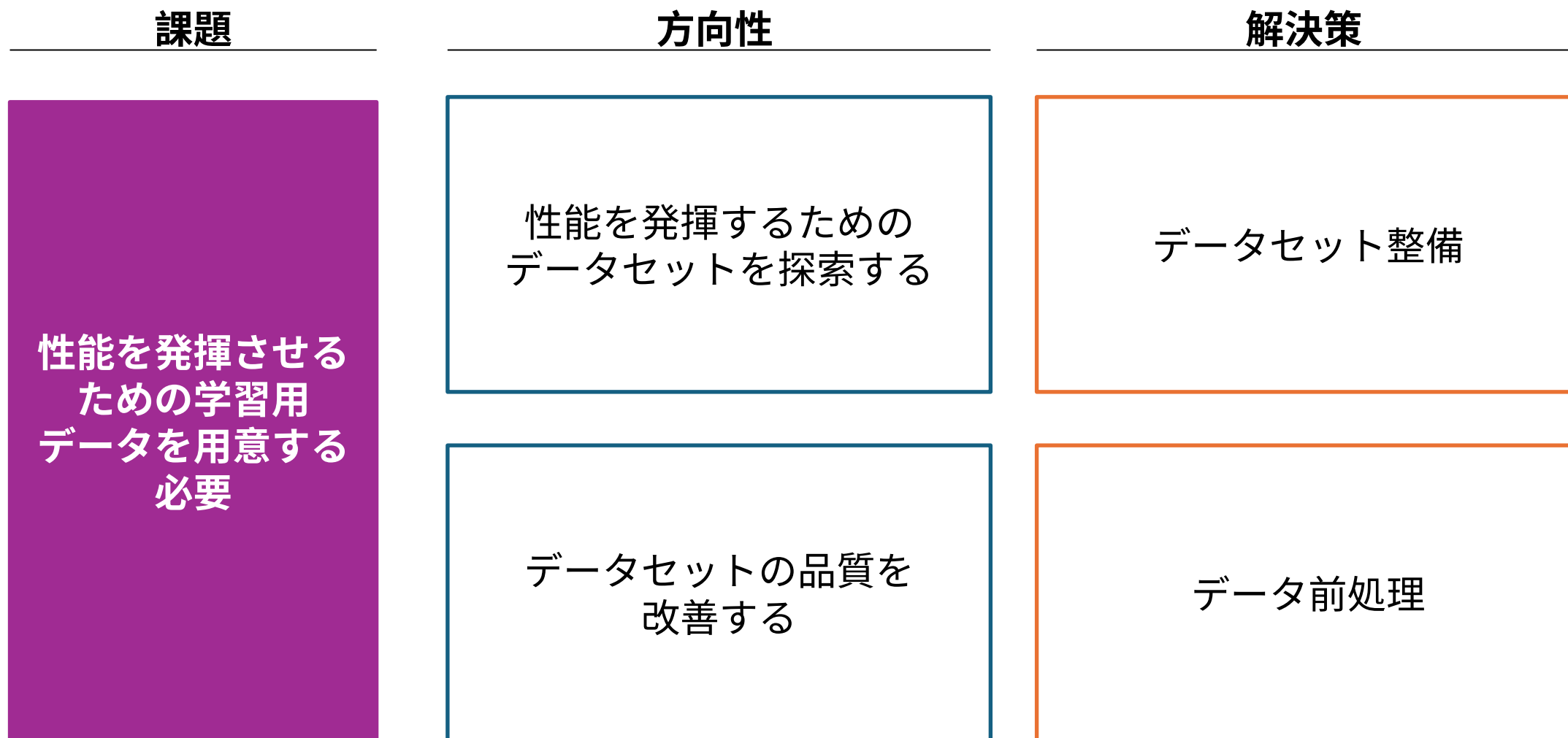
BitNet: パラメータを 2 値 ($\{-1, +1\}$; 1bit) / 3 値 ($\{-1, 0, 1\}$; 1.58bit) であらわ

- 量子化を適用したモデルを事前学習して構築 **す**
- 左図: Attention ・ MLP に含まれる線形層を全て 1bit 用に拡張された線形層 (BitLinear) に置き換えて構築
- 右図: モデルのビット数 (メモリ容量) で比較して既存の LLM の性能を大きく超えたことを報告

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- スケールするための技術: データ (D) に関連する取り組み

- それぞれの要素のスケールにおける問題点
- スケールするための技術: パラメータ数 (N) に関連する取り組み
- スケールするための技術: 計算量 (C) に関連する取り組み
- **スケールするための技術: データ (D) に関連する取り組み**

データセット (D) に関連する取り組みの全体像

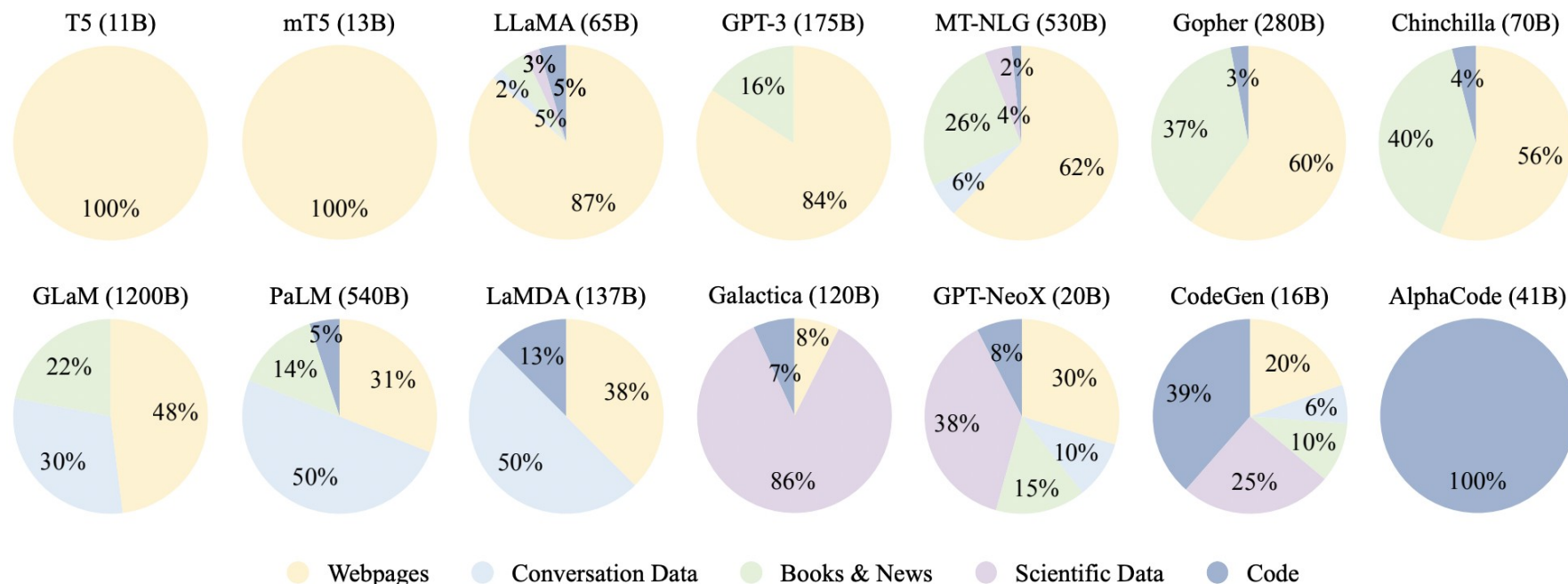


データセット (D) に関連する取り組み：データセット整備



どのような学習データで学習すべきか

■ 主要なモデルの学習データの構成



[2] Zhao+. A Survey of Large Language Models. 2023 より引用

- 最近のモデルは多くのケースで **Code** での学習を行っている． GPT-3 はなし．
- **Code** で学習したモデル（例： code-davinci-002 ）は GPT-3 より推論性能が良い

- ChatGPT も code-davinci-002 をベースに学習されているとされる。

特定ドメインのデータでの継続的な事前学習

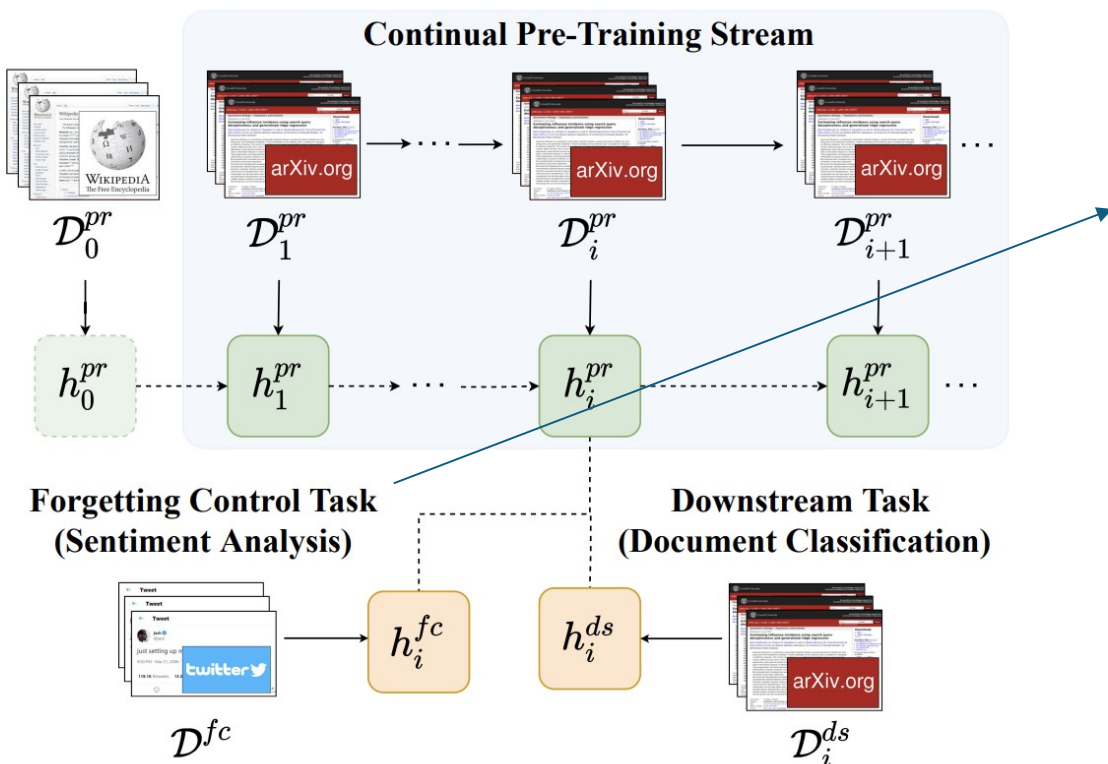


Table 2: Accuracy on the entire dataset of sentiment analysis with RoBERTa model. Continual pre-training has been performed sequentially over each experience of scientific abstracts. Base refers to the model pre-trained on Wikipedia, while NT refers to the model with vocabulary expansion.

RoBERTa		Accuracy					1-epoch Accuracy				
Base		93.40					92.40				
Exp.		e1	e2	e3	e4	e5	e1	e2	e3	e4	e5
Pretr		93.40	93.15	93.35	93.20	92.90	92.40	91.80	92.30	91.85	92.20
Pretr. NT		93.75	93.70	93.75	93.60	94.10	91.75	91.15	92.00	92.30	92.45

Table 10: Accuracy on the entire downstream dataset of scientific abstracts classification after joint training on the entire pre-training dataset of scientific abstracts. The *scratch* term indicates that the model is randomly initialized at the beginning and not pre-trained on Wikipedia.

Model	Accuracy	1-epoch Accuracy
RoBERTa Base	82.25	79.27
BERT Base	82.57	79.37
RoBERTa NT	81.84	77.88
RoBERTa Pr.	82.26	81.01
BERT Pr.	83.49	82.62
RoBERTa Pr. NT	83.51	81.94
RoBERTa scratch	80.48	75.79
RoBERTa scratch Pr.	82.50	81.50

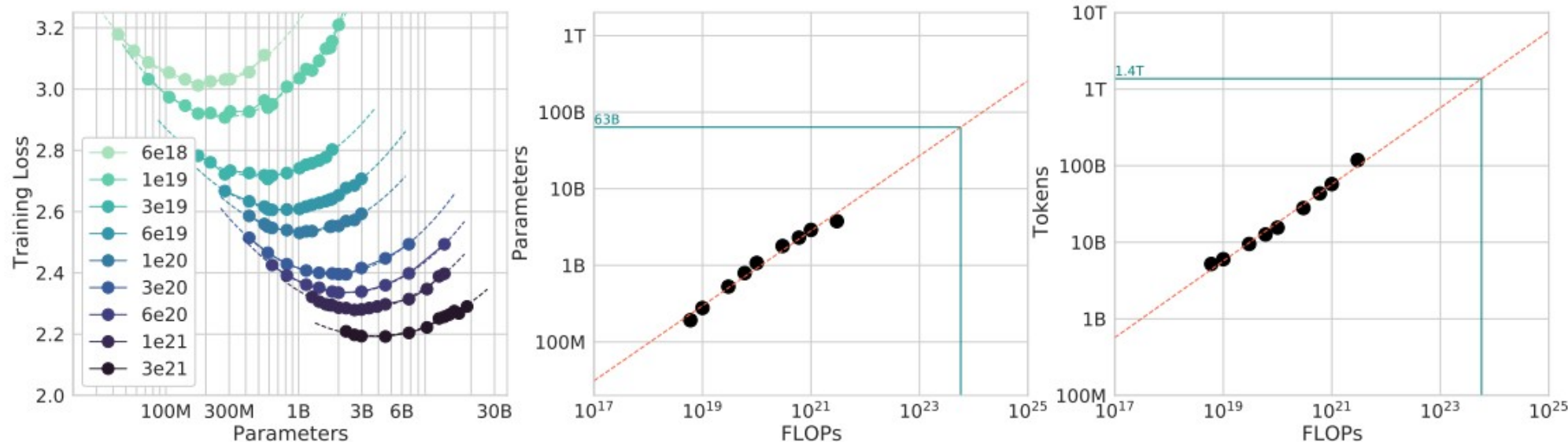
[41] Cossu+. Continual Pre-Training Mitigates Forgetting in Language and Vision. 2022 より引用

事前学習後に特定ドメインの文書 (e.g. arXiv の論文要旨) を継続的に学習させる。

継続学習したモデルの下流タスクでの性能を評価

事前学習の後に継続学習することで、**破滅的忘却が起きにくい上、下流タスクでの優れた性能を発揮できることを示す**

Chinchilla：最適計算配分に基づき N と D を決めたモデル



[42] Hoffmann+. Training Compute-Optimal Large Language Models. 2022 より引用

- LLM の事前訓練の予算は計算量（GPU 枚数や時間）に比例．
- 計算量をパラメータ数と学習データ量にどのように配分するかが重要となる．
- OpenAI による従来のスケール則 [3] はパラメータに対して必要となる **学習データ量の見積もりが少なすぎる** ことを指摘．

Chinchilla：最適計算配分に基づき N と D を決めたモデル



Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

データサイズ D

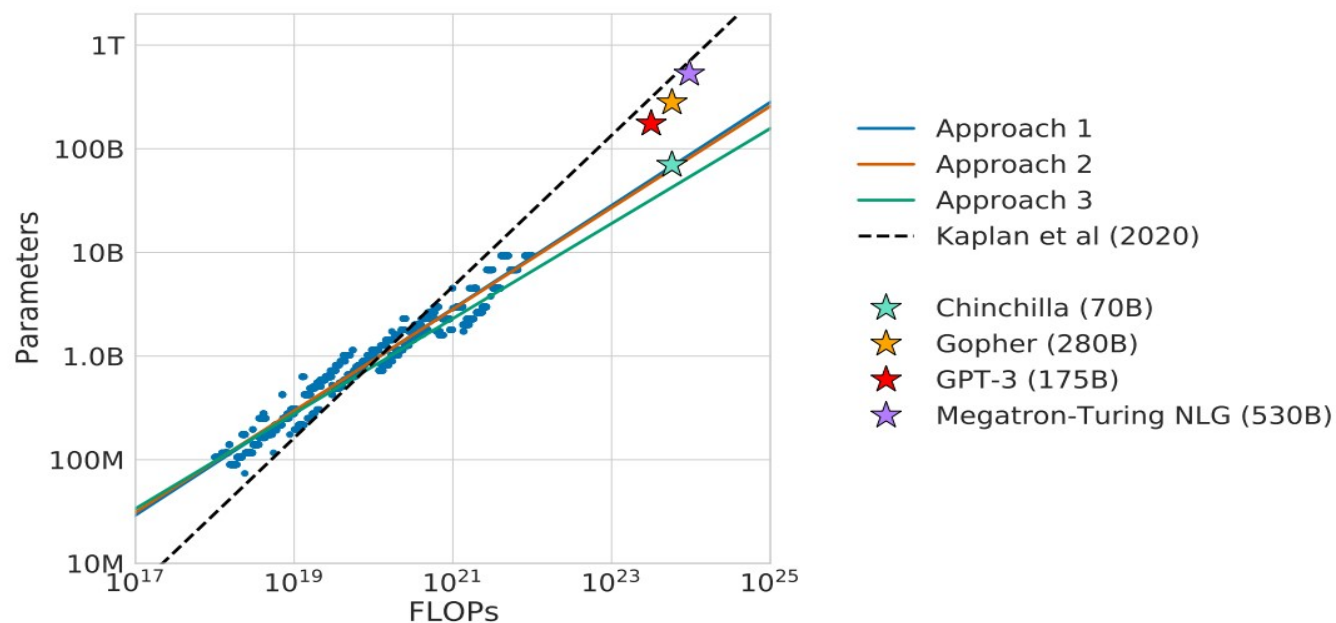
トークンを 1.4T まで増加
(同じデータの別サブセット)
※ Gopher の約 4.6 倍

モデルサイズ N

70B に設定
※ Gopher の約 1/4 倍

結果

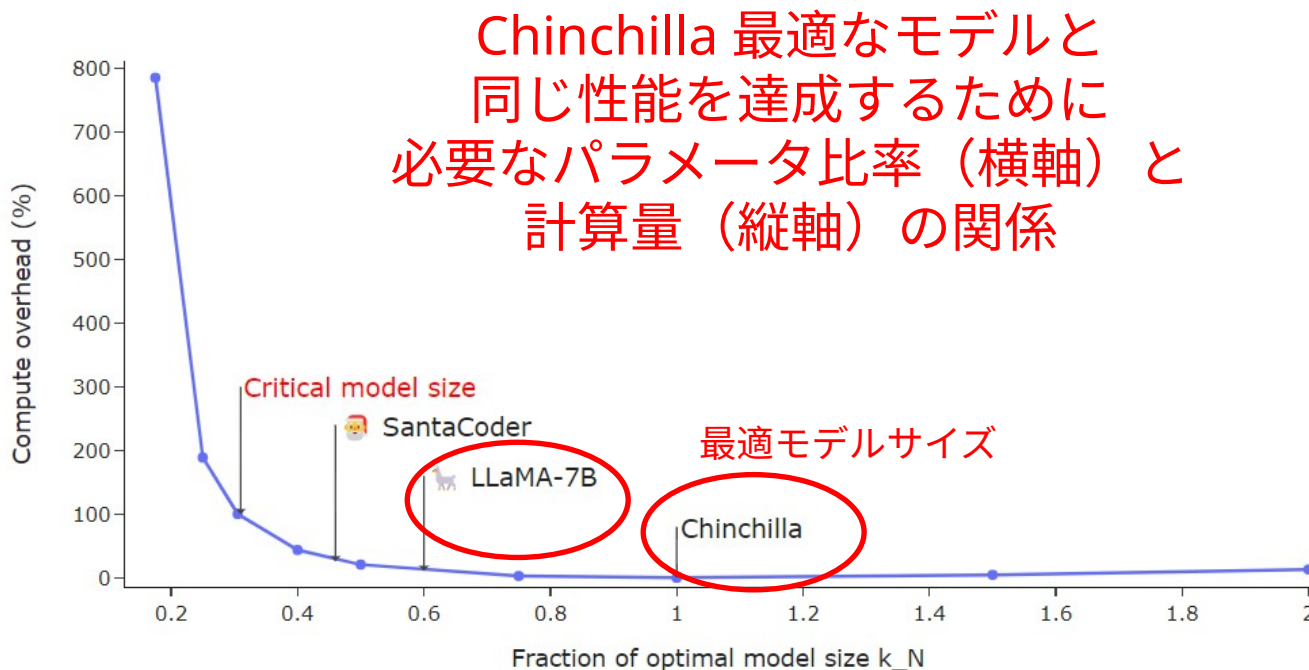
多くのケースで Gopher に勝利
(提案した関係式の妥当性を示唆)



[42] Hoffmann+. Training Compute-Optimal Large Language Models. 2022 より

引用 LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

実用的な小さいモデルでは、Chinchilla 則よりも多くのデータが必要？



- Chinchilla Trap:
Chinchilla のモデルサイズ (70B) は
大きいため、推論コストが高い*。
**推論コストも考慮してより
小さなモデルを大規模なデータで
訓練するべきではという意見**
- Chinchilla 最適なモデルサイズの
40-60% 以内のモデルサイズで、
10-42% の計算量の追加で同性能のモ
デルを学習できる

[43] Harm de Vries, Go smol or go home, Why we should train smaller LLMs on more tokens, 2023 より引用

“Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning”
[17]

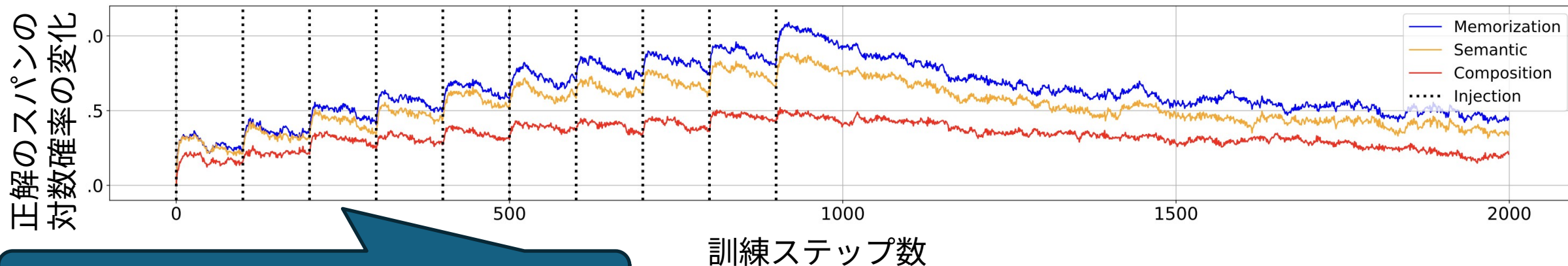
	Historical projection	Compute projection
Low-quality language stock	2032.4 [2028.4 ; 2039.2]	2040.5 [2034.6 ; 2048.9]
High-quality language stock	2024.5 [2023.55 ; 2025.75]	2024.1 [2023.2 ; 2025.3]
Vision stock	2046 [2037 ; 2062.8]	2038.8 [2032.0 ; 2049.8]

TABLE IV: Median and 90% CI of exhaustion year for each of the intersections.

過去の Web データの増え方，学習データの増え方からの予測
良質な言語データの枯渇が予測されている．

事前学習を通じて知識はどのように学習されていくのか

“How Do Large Language Models Acquire Factual Knowledge During Pretraining?” [60]



黒の点線の部分で知識を記述した文で訓練

Memorization: 注入した文をそのまま質問として利用

Semantic: 注入した文を言い換えた質問を利用

Composition: 複数の文の知識が必要な質問を利用

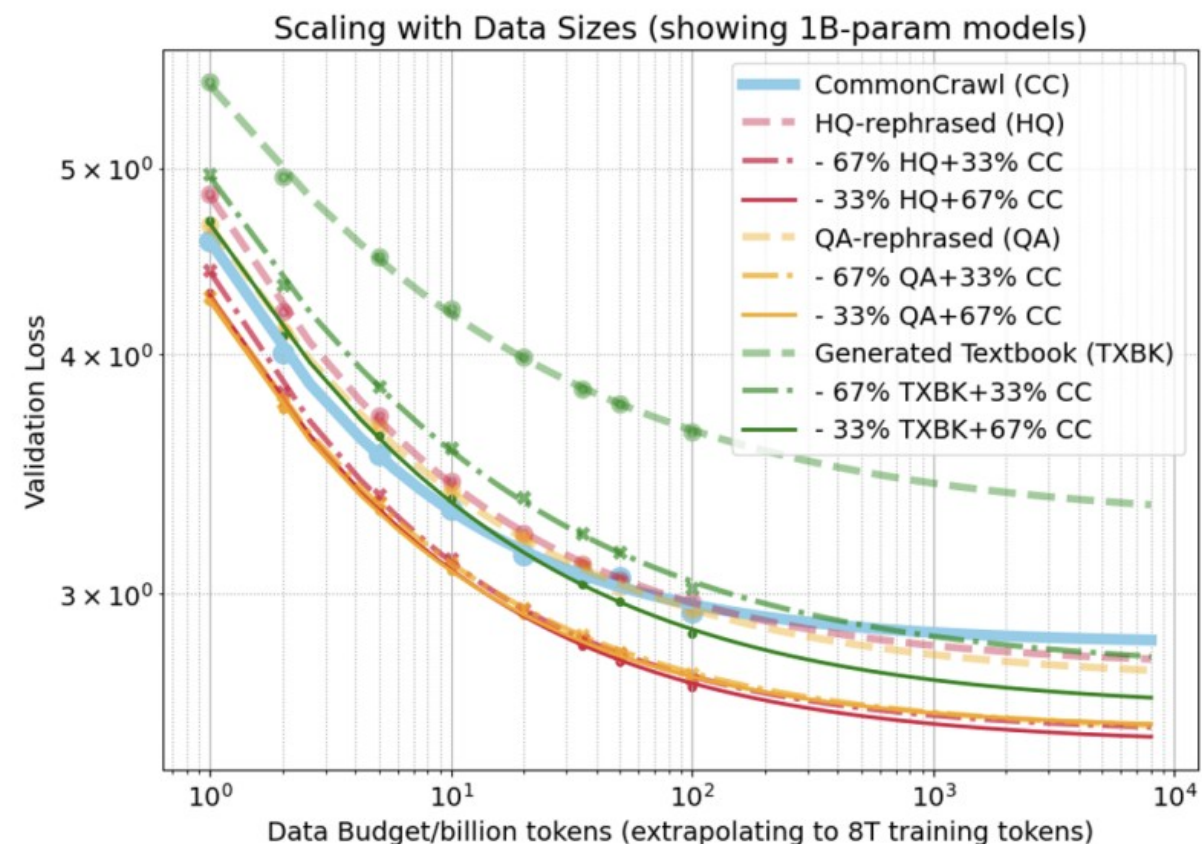
- LLM の事前訓練時に知識がどのように獲得されていくかを調査
- 知識を記述した文が出現する（点線）たびに正しい知識の生成される確率が高まり、知識が徐々に学習されていく
- 知識が出現しないステップ（900 ステップ以降）では、忘却されていく

LLM に知識を教えるためには、訓練データ中に
繰り返し知識が出現している必要がある



重要な知識が高い密度に含まれる高品質な訓練データの重要性を示唆

LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND



[59] Kang+. Demystifying Synthetic Data in LLM Pre-training: A Systematic Study of Scaling Laws, Benefits, and Pitfalls. 2025 より引用

方法

合成データを使った事前学習の効果を 1,000 以上の LLM を 10 万 GPU 時間を使って訓練して検証

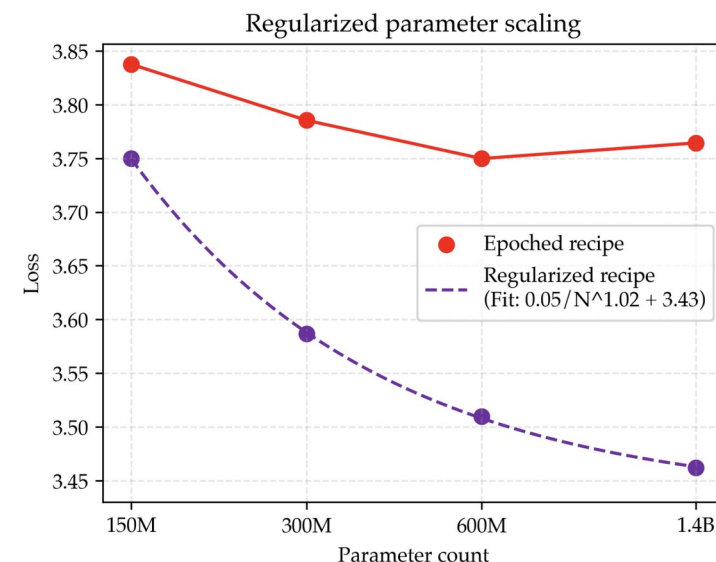
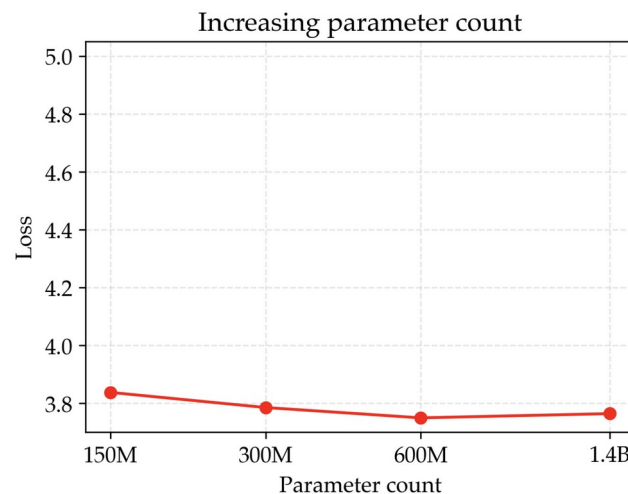
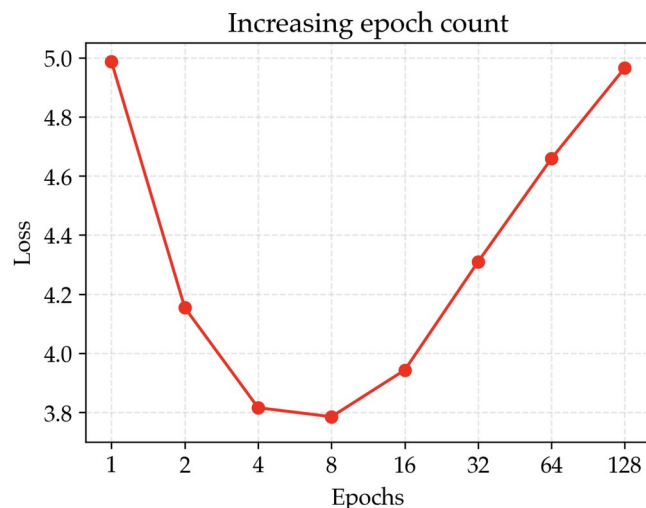
合成データの種類：

- **ウェブ言い換え**：LLM を使ったウェブデータの言い換えによる合成データ生成
 - **HQ**：綺麗なテキストへの言い換え
 - **QA**：QA への言い換え
- **TXBK**：合成テキストブック：LLM を使って教科書スタイルのデータを 0 から作成

結果

- **ウェブデータと合成データを混ぜた場合に訓練の効率が大幅に改善**
- 合成データのみで訓練すると性能悪化
→特に合成テキストブックのみでは顕著に悪化
- 全ての種類で、ウェブデータに 33% の比率で合成データを混ぜた場合に最善の性能を達成

データが有限で計算量が無限な設定での学習



標準的な訓練設定での学習では固定のデータ量で計算量だけ増やしてもスケールしない

[61] Kim+. Pre-training under infinite compute. 2025 より引用

- LLM の事前訓練に投下される計算量は年々増えているが、データは限られている
- データ量が有限であることを前提に、計算量をスケールする場合、性能を改善できるか？
- LLM の標準的な訓練設定ではスケールできない (左枠)**
エポック数を増やす → オーバフィットして性能低下 (左枠の左図)
モデルのサイズを増やす → 十分に訓練できず性能低下 (左枠の右図)
- 訓練設定を適切に調整すればスケールできる (右枠)**

訓練設定を適切に設定すればスケールできる

大きいモデルほど

- 小さい学習率
- 少ないエポック数
- 大きい weight decay (強い正則化)

データセット (D) に関連する取り組みの全体像



Table 1. **REFINEDWEB** improves on existing English pretraining datasets for large language models by combining extensive filtering with stringent deduplication at unprecedented scale. For additional details, see the full version in Table 12 of Appendix F.3.

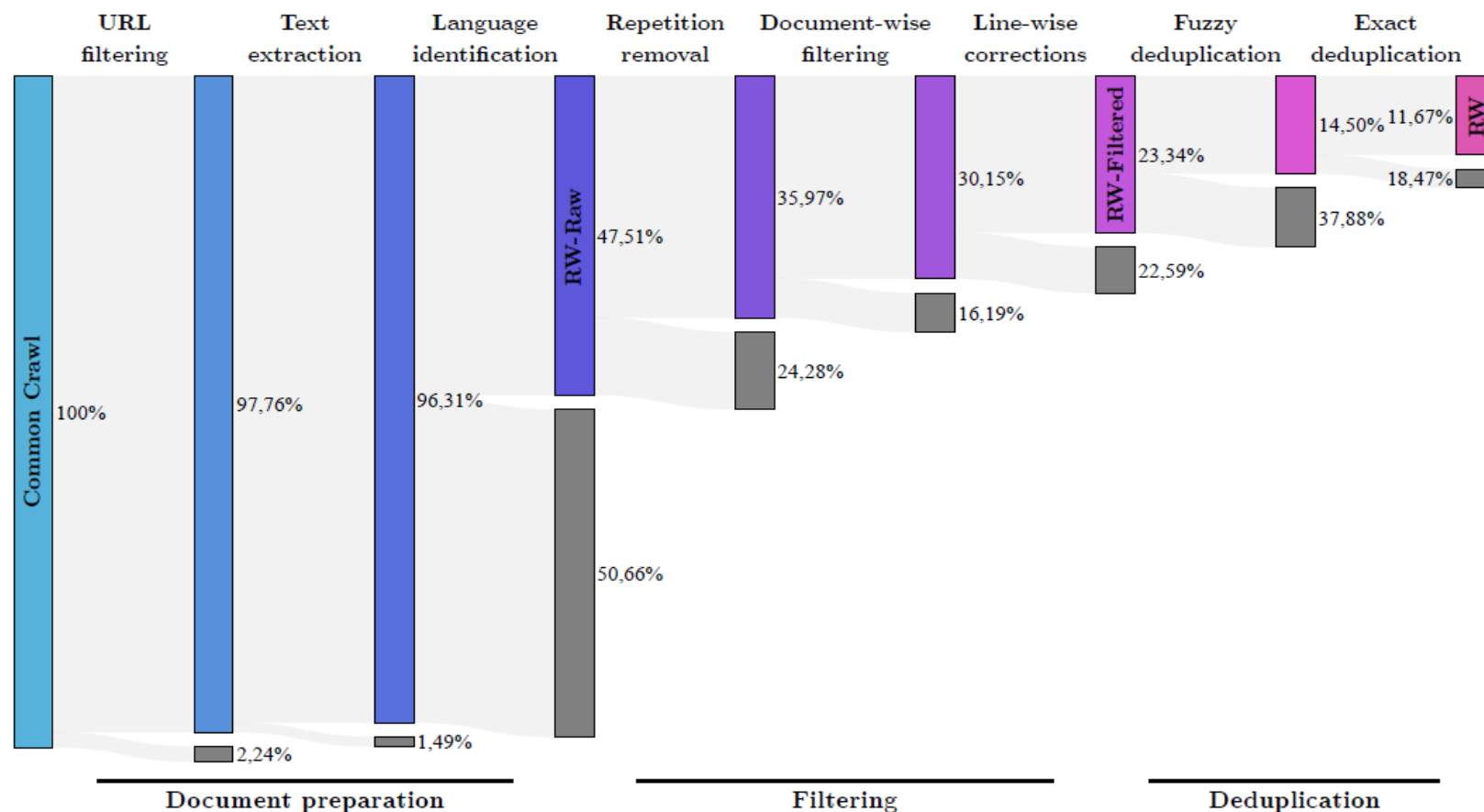
Dataset	Size	Availability	Web	CC Processing	Deduplication
MASSIVE WEB DATASETS					
C4	~ 360GT	Public	100%	Rules + NSFW words blocklist	Exact: spans of 3 sentences
OSCAR-21.09	~ 370GT	Public	100%	Built at the line-level	Exact: per line (~ 55% removed)
OSCAR-22.01	~ 283GT	Public	100%	Line-level rules + optional rules & NSFW URL blocklist	Exact: per line (optional, not used for results in this paper)
CURATED DATASETS					
■ GPT-3	300GT	Private	60%	Content filter trained on known high-quality sources	Fuzzy: MinHash (~ 10% removed)
▼ The Pile	~ 340GT	Public	18%	jusText for extraction, content filter trained on curated data	Fuzzy: MinHash (~ 26% removed)
★ PaLM	780GT	Private	27%	Filter trained on HQ data	Unknown
OURS					
●REFINEDWEB	~ 5,000GT	Public (600GT)	100%	trafilatura for text extraction, document and line-level rules, NSFW URL blocklist	Exact & fuzzy: exact substring+MinHash (~ 50% removed)

[39] Penedo+. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. 2023 より用

Web データのみでの 5T Token のデータセット。 600G が Public 。

フィルタリングの工夫 (後述) などにより以前より大規模なデータを構築。

Macrodata Refinement : データの厳密な絞り込みパイプライン



[39] Penedo+. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. 2023 より引用

- 複数のフィルタリング、重複削除を組み合わせた厳密なデータの絞り込みを実施
- 一連のパイプラインで CommonCrawl 中の約 90% の文書が取り除かれる

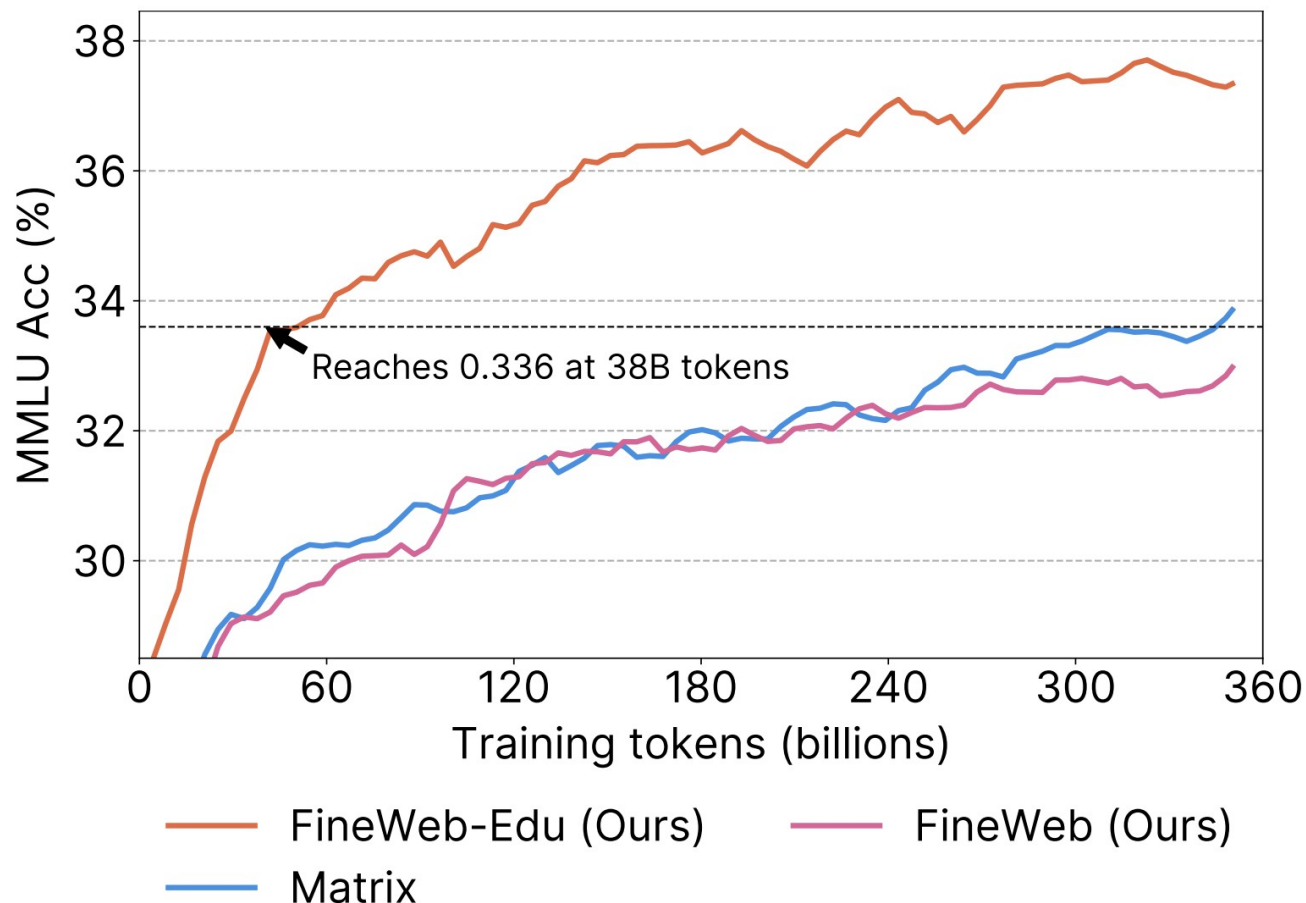
LLM 大規模言語モデル講座 講義資料 © 2025 by 東京大学松尾・岩澤研究室 is licensed under CC BY-NC-ND

4.0

- URL filtering: 有害な URL から取得したテキストを排除
- Text extraction: テキストのメインコンテンツテキストのみ抽出（ヘッダーや広告部分はいらない）
- Language identification: 特定の言語テキストのみ残す
- Repetition removal: テキスト内の繰り返し文を排除
- Document-wise filtering: スпамテキストをフィルタ
- Line-wise corrections: テキスト内の行レベルのフィルタ（例 SNS の「いいね」）
- Fuzzy deduplication: 異なるドキュメントに類似文章が存在した場合は排除 (MinHash^[40])
- Exact deduplication: 異なるドキュメントに指定したトークン数以上の完全一致が存在した場合は排除

[39] Penedo+. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. 2023 より引用

[40] Daisuke Okanohara, MinHash による高速な類似検索, Preferred Networks Research&Development, 2011 より引用



[62] Penedo+. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. 2024 より引用

FineWeb-Edu: LLM が評価したテキストの教育的価値を使って前処理を行う

方法

大規模テキストに LLM の推論を適用するのはコストが高いため、軽量化が必要。
→ LLM の評価結果を使って軽量分類器を訓練

- 46 万件のウェブ記事の「教育的価値」を LLM に評価させて訓練データを作成
- 小さいモデルを学習して評価器を作成

結果

- 前処理を行う前のデータ（FineWeb）や既存データ（Matrix）と比べて、知識や推論が必要なタスクの性能が大きく改善

モデルのスケールを支える技術動向について紹介しました。

1. なぜモデルをスケールさせるのか

1) スケール則の成立, 2) Emergent Ability

- ## 2. スケール則はモデルの性能と { パラメータ数, データ量, 計算量 } の関係を明らかにした
- スケール則で性能の予測ができるようになり, 大規模なモデルへの投資リスクが軽減.

3. モデルをスケールする上での問題は依然として存在

- スケールにつれて必要となるコストの増加, データの不足, etc..

4. モデルのスケールを支える様々な研究・開発が行われている

- パラメータ数 (P): よりメモリ効率, 演算効率の優れたモデルの提案
- 計算量 (C): 効率的な学習、推論方法の整備
- データセットサイズ (D): データの量と質の工夫

- [1] Bao Hua Choo, The emergence of Large Language Models (LLMs), The low down, <https://thelowdown.momentum.asia/the-emergence-of-large-language-models-llms/>, アクセス日 : 2023/11/16
- [2] Zhao+. A Survey of Large Language Models. 2023. In arXiv:2303.18223
- [3] Kaplan+. Scaling Laws for Neural Language Models. 2020. In arXiv:2001.08361
- [4] Wei+. Emergent Abilities of Large Language Models. 2022. In arXiv:2206.07682
- [5] Schaeffer+. Are Emergent Abilities of Large Language Models a Mirage?. 2023. In arXiv:2304.15004
- [6] Power+. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. 2022. In arXiv:2201.02177
- [7] Liu+. Towards Understanding Grokking: An Effective Theory of Representation Learning. 2022. In NeurIPS2022
- [8] Hestness+. Deep Learning Scaling is Predictable, Empirically. 2017. In arXiv:1712.00409
- [9] Brown+. Language Models are Few-Shot Learners. 2020. In NeurIPS2020
- [10] Anil+. PaLM 2 Technical Report. 2023. In arXiv:2305.10403

- [11] Henighan+. Scaling Laws for Autoregressive Generative Modeling. 2020. In arXiv:2010.14701
- [12] Ganguli+. Predictability and Surprise in Large Generative Models. 2023. In arXiv:2202.07785
- [13] OpenAI. GPT-4 Technical Report. 2023. In arXiv:2303.08774
- [14] Abhinav Venigalla, Linden Li, Billion-Parameter GPT Training Made Easy, MosaicML, <https://www.mosaicml.com/blog/billion-parameter-gpt-training-made-easy>, アクセス日 : 2023/11/16
- [15] Vaswani+. Attention Is All You Need. 2017. In NeurIPS2017
- [16] Jaiyam Sharma, Understanding Attention Mechanism in Transformer Neural Networks, LearnOpenCV, <https://learnopencv.com/attention-mechanism-in-transformer-neural-networks/>, アクセス日 : 2023/11/16
- [17] Villalobos+. Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning. 2022. In arXiv:2211.04325
- [18] Tay+. Efficient Transformers: A Survey. 2020. In arXiv:2009.06732
- [19] Child+. Generating Long Sequences with Sparse Transformers. 2019. In arXiv:1904.10509
- [20] Zahher+. Big Bird: Transformers for Longer Sequences. 2020. In NeurIPS2020

- [21] Dao+. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. In NeurIPS2022
- [22] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. In arXiv:2307.08691
- [23] Chen+. Towards Understanding Mixture of Experts in Deep Learning. 2022. In NeurIPS2022
- [24] Shazeer+. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. 2017. In ICLR
- [25] Fedus+. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. 2021. In arXiv:2101.03961
- [26] Rajbhandari+. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. 2022. In ICML2022 Proceedings of the 39th International Conference on Machine Learning, PMLR 162:18332-18346
- [27] Clark+. Unified Scaling Laws for Routed Language Models. 2022. In arXiv:2202.01169
- [28] Zhai+. An Attention Free Transformer. 2021. In arXiv:2105.14103
- [29] Peng+. RWKV: Reinventing RNNs for the Transformer Era. 2023. In arXiv:2305.13048
- [30] Sun+. Retentive Network: A Successor to Transformer for Large Language Models. 2023. In arXiv:2307.08621

- [31] Gu+. Efficiently Modeling Long Sequences with Structured State Spaces. 2022. In ICLR2022
- [32] Microsoft DeepSpeed Team, DeepSpeed: Extreme-scale model training for everyone, Microsoft, <https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/>, アクセス日 : 2025/10/05
- [33] Rajbhandari+. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. 2019. In arXiv:1910.02054
- [34] Microsoft, <https://github.com/microsoft/DeepSpeed>, アクセス日 : 2023/11/16
- [35] DeepSpeed, <https://www.deepspeed.ai/docs/config-json/>, アクセス日 : 2023/11/16
- [36] Younes Belkada, Tim Dettmers, A Gentle Introduction to 8-bit Matrix Multiplication for transformers at scale using Hugging Face Transformers, Accelerate and bitsandbytes, Hugging Face, <https://huggingface.co/blog/hf-bitsandbytes-integration#a-gentle-introduction-to-8-bit-matrix-multiplication-for-transformers-at-scale-using-hugging-face-transformers-accelerate-and-bitsandbytes>, アクセス日 : 2023/11/16
- [37] Dettmers+. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. 2022. In NeurIPS2022
- [38] Liu+. Do Emergent Abilities Exist in Quantized Large Language Models: An Empirical Study. 2023. In arXiv:2307.08072
- [39] Penedo+. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. 2023. In arXiv:2306.01116
- [40] Daisuke Okanohara, MinHash による高速な類似検索 , Preferred Networks Research&Development, 2011, <https://tech.preferred.jp/ja/blog/minhash/>, アクセス日 : 2023/11/16

- [41] Cossu+. Continual Pre-Training Mitigates Forgetting in Language and Vision. 2022. In arXiv:2005.09357
- [42] Hoffmann+. Training Compute-Optimal Large Language Models. 2022. In NeurIPS2022
- [43] Harm de Vries, Go smol or go home, Why we should train smaller LLMs on more tokens, 2023, <https://www.harmdevries.com/post/model-size-vs-compute-overhead/>, アクセス日 : 2023/11/16
- [44] Sorscher+. Beyond neural scaling laws: beating power law scaling via data pruning. 2022. In NeurIPS2022
- [45] Tirumala+. D4: Improving LLM Pretraining via Document De-Duplication and Diversification. 2023. In arXiv:2308.12284
- [46] Zhou+. LIMA: Less Is More for Alignment. 2023. In arXiv:2305.11206
- [47] Dzmitry Bahdanau, The FLOPs Calculus of Language Model Training, Medium, 2022, <https://medium.com/@dzmitrybahdanau/the-flops-calculus-of-language-model-training-3b19c1f025e4>, アクセス日 : 2023/11/16
- [48] Wan+. Efficient Large Language Models: A Survey. 2024. In arXiv:2312.03863
- [49] Patro and Agneeswaran. Mamba-360: Survey of State Space Models as Transformer Alternative for Long Sequence Modelling: Methods, Applications, and Challenges. 2024. In arXiv:2404.16112
- [50] De+. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. 2024. In arXiv:2402.19427

- [51] Qu+. A Survey of Mamba. 2024. In arXiv:2408.01129
- [52] Feng+. Beyond Model Collapse: Scaling Up with Synthesized Data Requires Reinforcement. 2024. In arXiv:2406.07515
- [53] Gerstgrasser+. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. 2024. In arXiv:2404.01413
- [54] Munkhdalai+. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. 2024. In arXiv:2404.07143
- [55] NVIDIA NeMo Framework User Guide - Parallelisms, NVIDIA, <https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/features/parallelisms.html>, アクセス日 : 2025/10/05
- [56] Dettmers+. The case for 4-bit precision: k-bit Inference Scaling Laws. 2023. In arXiv:2212.09720
- [57] Wang+. BitNet: Scaling 1-bit Transformers for Large Language Models. 2023. In arXiv:2310.11453
- [58] Ma+. BitNet b1.58 2B4T Technical Report. 2025. In arXiv:2504.12285
- [59] Kang+. Demystifying Synthetic Data in LLM Pre-training: A Systematic Study of Scaling Laws, Benefits, and Pitfalls. 2025. In arXiv:2510.01631
- [60] Chang+. How Do Large Language Models Acquire Factual Knowledge During Pretraining?. 2024. In arXiv:2406.11813

- [61] Kim+. Pre-training under infinite compute. 2025. In arXiv:2509.14786
- [62] Penedo+. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. 2024. In arXiv:2406.17557
- [63] Younes Belkada, Tim Dettmers, A Gentle Introduction to 8-bit Matrix Multiplication for transformers at scale using Hugging Face Transformers, Accelerate and bitsandbytes, Hugging Face, <https://huggingface.co/blog/hf-bitsandbytes-integration#a-gentle-summary-of-llm-int8-zero-degradation-matrix-multiplication-for-large-language-models>, アクセス日 : 2026/05/25
- [64] sunbluesome. Sparse Transformer を理解したい , Zenn, <https://zenn.dev/sunbluesome/articles/5f6a86dfa1e1be>, アクセス日 : 2023/11/16
- [65] weights & biases, 「LLM をゼロからトレーニングするためのベストプラクティス」 , <https://wandb.ai/site/resources/whitepapers/llm-whitepaper-japan/> アクセス日 : 2026/05/25
- [66] iwiwi, github, <https://gist.github.com/iwiwi/fc174b1f2341c2c0170be87c5b2e1d31>, アクセス日 :2026/05/25

